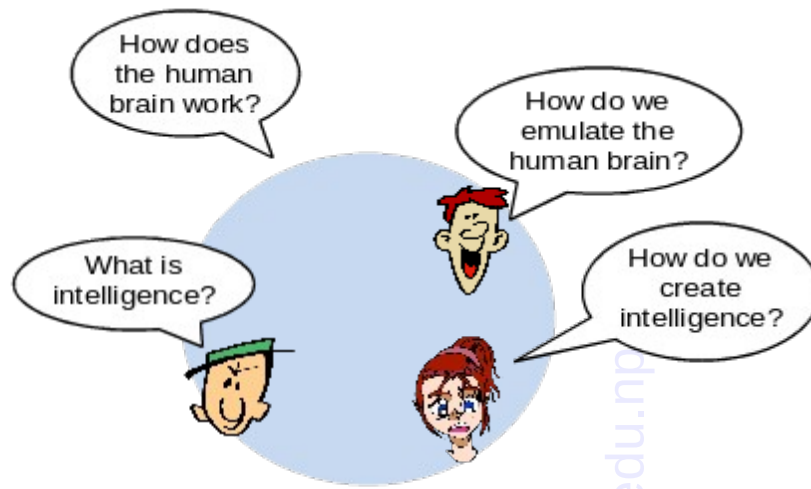


Chapter -1: Introduction to Artificial Intelligence



Intelligent behaviors

- **Everyday tasks:** recognize a friend, recognize who is calling, translate from one language to another, interpret a photograph, talk, cook a dinner
- **Formal tasks:** prove a logic theorem, geometry, calculus, play chess, checkers, or Go
- **Expert tasks:** engineering design, medical designers, financial analysis

Intelligence is:

- The ability to reason
- The ability to understand
- The ability to create. . .

Can we produce a machine with all these abilities? The answer is no, so then what is AI?

Artificial Intelligence

“The art of creating machines that perform functions that require intelligence when performed by people” – Kurzweil, 1990

“The science of making computers, do things that require intelligence like humans”- Minsky

“AI is the study of how to make computers do things at which, at the moment, people are better”- Elaine Rich

AI Today

- Diagnose lymph- node diseases
- Monitor space shuttle missions
- Automatic vehicle control
- Large- scale scheduling
- Detection of money laundering
- Classify astronomical objects

- Speech understanding systems
- Beat world's best players in chess, checkers, and backgammon.

AI Topics

- Robotics
- Search
- Planning
- Machine learning
- Image processing
- Expert systems
- Natural Language Processing

Example Eliza

- **ELIZA:** A program that simulated a psychotherapist interacting with a patient and successfully passed the Turing Test.
- **Coded at MIT during 1964- 1966 by Joel Weizenbaum.**
 - First script was DOCTOR.
 - The script was a simple collection of syntactic patterns not unlike regular expressions
 - each pattern had an associated reply which might include bits of the input (after simple transformations (my -> your))
 - Weizenbaum was shocked at reactions:
 - Psychiatrists thought it had potential.
 - People unequivocally anthropomorphised.

Is AI Ethical

- Joseph Weizenbaum (1976) in *Computer Power and Human Reason* argues:
- A *real* AI would indeed be an autonomous, intelligent agent
 - Hence, out of our control
- It will not share our: motives, constraints, ethics
- There is no obvious upper bound on intelligence. And perhaps there is no upper bound at all.
 - When our interests and AI's interests conflict, guess who loses
 - Therefore, AI research is unethical.

Asimov's laws of Robotics

- A method to insert ethics into AI
- The three laws of robots are:
 - A robot may not injure a human being or, through inaction, allow a human being to come to harm.

- A robot must obey the orders given it by human beings.
- A robot must protect its own existence.
- **Meta-law:** Precedence order is lower to higher

Brief history

- 1943 McCulloch & Pitts: Boolean circuit model of brain
- 1950 Turing's "Computing Machinery and Intelligence"
- 1956 Dartmouth meeting: "Artificial Intelligence" adopted
- 1950s Early AI programs, including Samuel's checkers program, Newell & Simon's Logic Theorist, Gelernter's Geometry Engine
- 1965 Robinson's complete algorithm for logical reasoning
- 1966—73 AI discovers computational complexity, neural network research almost disappears
- 1969—79 early development of knowledge-based systems
- 1980-- AI becomes an industry
- 1986-- Neural networks return to popularity
- 1987-- AI becomes a science
- 1995-- The emergence of intelligent agents

Applications of AI

- **Game playing:** IBM's Deep Blue became the first computer program to defeat the world champion in a chess match when it bested Garry Kasparov by a score of 3.5 to 2.5 in an exhibition match in 1997.
- **Autonomous control:** The Alvin computer vision system was trained to steer a car to keep it following a lane. It was placed in CMU's NavLab computer-controlled minivan and used to navigate across the United States—for 2850 miles it was in control of steering the vehicle 98% of the time. A human took over the other 2%, mostly at exit ramps. NavLab has video cameras that transmit road images to Alvin, which then computes the best direction to steer, based on experience from previous training runs.
- **Language understanding and problem solving:**
- **Business Intelligence**
- **Medical Diagnosis**
- **Scientific Analysis**
- **Weather forecasting**
- **And many more**

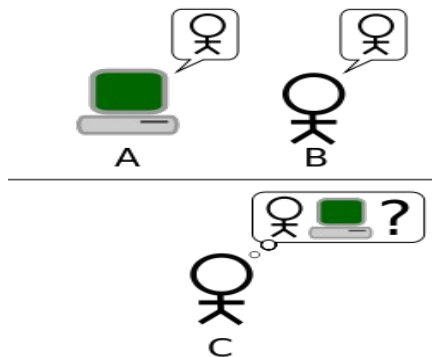
Note: Describe the application with few sentences:

Approaches of AI

- Acting humanly
- Thinking humanly
- Acting rationally
- Thinking rationally

Acting humanly

- Machine is required to act as humans do.
- **Turing Test**, proposed by Alan Turing (1950). Turing's definition of intelligent behavior as the ability to **achieve human-level performance** in all cognitive tasks.
- The test involves an interrogator who interacts with one human and one machine. Within a given time the interrogator has to find out which of the two the human is, and which one the machine.



- Issue to solve in order to pass the test using a terminal: natural language processing, knowledge representation automated reasoning, machine learning.
- Additional requirements for the “total Turing test”: computer vision, speech recognition, speech synthesis, robotics.

Critics of Turing test

- Test is not reproducible, amenable or constructive to mathematical analysis as it is more important to study the underlined principles of intelligence than to duplicate example
- Trying to evaluate machine intelligence in terms of human intelligence is fundamental mistake. It focuses too much on the behavior of conversation

Factors required to pass Turing Test

- . **Natural Language Processing:** To communicate easily.
- . **Knowledge Representation:** To store facts and rules.
- . **Automated Reasoning:** To draw conclusion from stored knowledge.
- . **Machine Learning:** To adopt new circumstances and detect pattern.

Thinking humanly

- It is the action or process of acquiring knowledge and understanding thought, experience or the sense. Machine needs to understand how humans think.
- It is mainly concerned with the investigation, development of human behavior and the working of human mind in the computer.
- **Example: General Problem Solver (GPS)** developed by Newell & Simon in 1961 attempted to synthesize the human solving process. Compares the steps of the problem with the reasoning of human subjects solving the same problem.
- **Critics:** Lacks scientific theories of internal activities of brain. Level of abstractin of the approach is vague too.

Thinking rationally

- Aristotle made first attempt to harness “right thinking”. Famous example: “Socrates is a man; all men are mortal; therefore Socrates is mortal.”
- Formal logic (late nineteenth, early twentieth century’s) provides a precise notation of statements of all kinds of things and relations between them.
- Programs that can find the solution to a logical problem if one exists have been around since 1965. Given enough memory and time, they should be able to solve everything.
- **Critics:** It is not easy to take informal knowledge and state in the formal terms required by logical notation specially when knowledge is less than 100% certain
- There is big difference between being able to solve a problem in principle and doing so in practice

Acting rationally (Rational agent approach)

- Rational agent: acts to achieve the best outcome on when there is uncertainty of the best expected outcome
 - Rational agent does not necessarily involve thinking. Example, reflex action
- Advantages
 - More general than laws of thought approach
 - More amenable to scientific development then approach based on human behaviors.

Chapter -2: Problem Solving

- Problem solving is a agent based system that finds sequence of actions that lead to desirable states from the initial state.
- Four steps of problem solving are:
 - i. Goal Formulation: Helps to organize behavior by isolating and representing the task knowledge necessary to solve problem.
 - ii. Problem Formulation: Define the problem precisely with initial states, final state and acceptable solutions.
 - iii. Searching: Find the most appropriate techniques o sequence among all possible techniques.
 - iv. Execution: Once the search algorithm returns a solution to the problem, the solution is then executed by the agent.

Techniques

State Space Representation

- A state space essentially consists of a set of nodes representing each state of the problem, arcs between nodes representing the legal moves from one state to another, an initial state and a goal state.
- A problem can be defined by
 - . Initial state
 - . Actions (Using successor function)
 - . Goal test (to determine the goal state)
 - . Path cost
- A problem is when defined with these components is called **well defined problem**.
- The actions and rules should be defined in as general way as possible. If the specific rules are made, the rule set becomes very large.
Eg; Chess world.

8								
7		H		H				
6	H				H			
5			H					
4	H				H			
3		H		H				
2								
1								
	1	2	3	4	5	6	7	8

Let Current Position of Horse: (4,4)

Rules for next possible position can be as: (2,4),(2,6), (3,3),(3,7),(5,3)(5,7),(6,4) and (6,6)

- When the current position of the horse changes from (4,4) to any other location, we have to define another rule to find next possible position. By doing so the rule set becomes very large.

- There for specific pattern should be described such as for above case the pattern rule can be as:
Next possible Position: Current Position +/- ((2 vertical + 1 Horizontal) or (1 vertical + 2 Horizontal)) position.
- When a problem is defined with all its states, it is said to be in **complete state space**.

Examples:

1. **Vacuum World**
 2. **Water Jug Problem**
 3. **Tower of Hanoi**
- Etc.

(Refer class note for solution)

Production System

- A production system consists of a set of rules, each consisting of a left hand side (pattern) that determines the applicability of rules and a right side that describes the operation to be performed if the rule is applied.
Eg. [A, clean] => move right
 Pattern Action (Operation)
 [A, dirt] => clean
- A production system may have one or more knowledgebase that contain whatever information is appropriate for the particular task.
- A control strategy that specifies the order in which the rules will be selected and a way of resolving the conflicts that arise when several rules matched at once i.e. it must have a **Rule Applier** for conflict resolution.
- The first requirement of a good control strategy is that it must cause motion.
- The second requirement of a good control strategy is that it should have systematic.

Evolution Function

- Evaluation function evaluates any state or node that gives a number to indicate how far we are from the goal.
- Every move should reduce this number or if not never increase.
- When this number becomes zero, the problem is solved.
- **Eg. 8-Puzzle game**
(Refer class note)

Problem Classification

- i. Ignorable:** Intermediate actions can be ignored. Eg. Water-jug problem.
- ii. Recoverable:** The actions can be implemented to go the initial state. Eg. 8-puzzle game.
- iii. Irrecoverable:** The actions cannot help to reach the precious state. Eg. Tic-tac-toe.
- iv. Decomposable:** The problem can be broken into similar ones. Eg. Multiplayer game.

Constraints Satisfaction Problem

- A search procedure that operates in a space of constraints.
- Constraints are discovered and propagated as far as possible throughout the system.
- A guess about something is made and added as a new constraint.
- The problem can be described as a set of variables ($X_1, X_2, \dots$) and Constraints ($C_1, C_2, \dots$)
- Constraint propagation terminates for one of two reasons.
 - i. Contradiction detected i.e. no solution consistent with known constraints.
 - ii. Propagation has run off stream and there are no further changes that can be made on the basis of current knowledge.

Eg. Crypto-arithmetic

- i. **SEND + MORE = MONEY**
- ii. **RIGHT + RIGHT = WRONG**
- iii. **WRONG + WRONG = RIGHT**
- iv. **LOGIC + LOGIC = PROLOG**
- v. **CROSS + ROADS = DANGER**
- vi. **BASE + BALL = GAMES**
- vii. **ONE + ONE = TWO**
- viii. **ONE + ONE + TWO = FOUR**
- ix. **KYOTO + OSAKA = TOKYO**
- x. **APPLE + GRAPE = CHERRY**

Chapter- 3: Searching

- Searching the process finding the required states or nodes.
- Searching is to be performed through the state space.
- Search process is carried out by constructing a search tree.
- Search is a universal problem-solving technique.
- Search involves systematic trial and error exploration of alternative solutions.
- Many problems don't have a simple algorithmic solution. Casting these problems as search problems is often the easiest way of solving them.
- Useful when the sequence of actions required to solve a problem is not known
 - o Path finding problems, e.g, eight puzzle, travelling salesman problem
 - o Two player games, e.g., chess and checkers
 - o Constraint satisfaction problems, e.g., eight queens

Steps in Searching

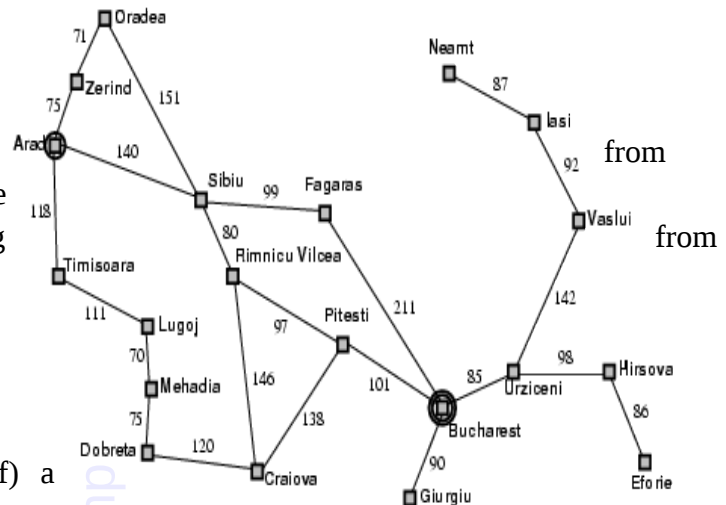
1. Check whether the current state is the goal state or not?
2. Expand the current state to generate the new sets of states.
3. Choose one of the new states generated for search depending upon search strategy.
4. Repeat step 1 to 3 until the goal state is reached or there are no more state to be expanded.

Problem Formulation

- A search problem is defined in terms of states, operators and goals
- A state is a complete description of the world for the purposes of problem-solving
 - o Initial state is the state the world is in when problem solving begins
 - o Goal state is a state in which the problem is solved
- An operator is an action that transforms one state of the world into another state
- Goal states
 - o Depending on the number of solutions a problem has, there may be a single goal state or many goal states:
 - o In the eight-puzzle there is a single solution and a single goal state
 - o In chess, there are many winning positions, and hence many goal states
- Applicable operators
 - o In general, not all operators can be applied in all states
 - o In a given chess position, only some moves are legal (as defined by the rules of chess)
 - o In a given eight-puzzle configuration, only some moves are physically possible
 - o The set of operators which are *applicable in a state s* determine the states that can be reached from s

Example : Route Planning

- State: Being in any one city
- Initial State: Being in **Arad**
- Goal State: Be in **Bucharest**
- Operators: actions of driving city X to city Y along the connecting roads, e.g, driving Arad to Sibiu



State vs Nodes

- A state is a (representation of) a physical configuration
- Nodes in the search tree are data structures maintained by a search procedure representing *paths to a particular state*
- The same state can appear in several nodes if there is more than one path to that state
- The path can be reconstructed by following edges back to the root of the tree

Search strategies

- A search strategy is defined by picking the order of node expansion
- Strategies are evaluated along the following dimensions:
 - o **Completeness**: does it generate to find a solution if there is any?
 - o **Optimality**: does it always find the highest quality (least-cost) solution?
 - o **Time complexity**: How long does it take to find a solution?
 - o **Space complexity**: How much memory does it need to perform the search?
- Time and space complexity are measured in terms of
 - b : maximum branching factor of the search tree
 - d : depth of the least-cost solution
 - m : maximum depth of the state space (may be ∞)

Types of Search

- Uninformed (Blind) Search
- Informed (Heuristic) Search

Uninformed search strategies

- These types of search strategies are provided with the problem definition and these don't have additional information about the state space.
- These can only expand current state to get a new set of states and distinguish a goal state from non-goal state.
- Uninformed search strategies use only the information available in the problem definition
- Less effective than informed search.
- Distinguish goal state from non goal state
 - Breadth-first search

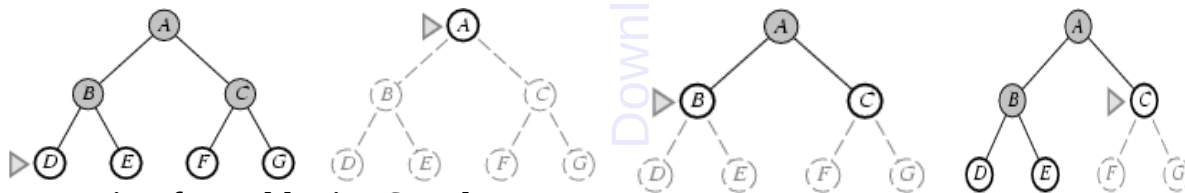
- Uniform cost search
- Depth-first search
- Depth-limited search
- Iterative Deepening Depth-first search
- Bidirectional Search

Breadth-First Search (BFS)

- Proceeds level by level down the search tree
- Starting from the root node (initial state) explores all children of the root node, left to right
- If no solution is found, expands the first (leftmost) child of the root node, then expands the second node at depth 1 and so on ...
- Process
 - Place the start node in the queue
 - Examine the node at the front of the queue
 - If the queue is empty, stop
 - If the node is the goal, stop
 - Otherwise, add the children of the node to the end of the queue

BFS example (Find path from A to D)

Put the start node in the queue, Examine the first element of the queue. If it is the goal, stop, otherwise put its children in the queue



Properties of Breadth-First Search

- **Completeness:** Complete if the goal node is at finite depth
- **Optimality:** It is guaranteed to find the shortest path
- **Time complexity**
 - For branching factor b and depth level d
 - Expand root yields b nodes at 1st level
 - Expand 1st level yields b^2 nodes at 2nd level.
 - If the goal is in d^{th} level, in the worst case, the goal node would be the last node in the d^{th} level.

- We should expand $(b^d - 1)$ nodes in the d^{th} level (except the goal node itself). Total nodes in d^{th} level = $b(b^d - 1) = b^{d+1} - b$
- Total no of nodes generated = $1 + b + b^2 + b^3 + \dots + b^{d+1} - b$
- Time complexity = $O(b^{d+1})$
- **Space Complexity:** $O(b^{d+1})$

Weakness of BFS

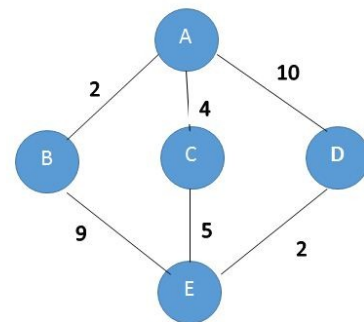
- *High time and memory requirements*

Uniform Cost Search

- Breadth first Search finds the shallowest goal but it's not always sure to find the optimal solution.
- Uniform cost search can be used if the cost of travelling from one node to another is available.
- Uniform cost search always expands the lowest cost node on the fringe (the collection of nodes that are waiting to be expanded.)
- The first solution is guaranteed to be the cheapest one because a cheaper one is expanded earlier and so would have been found first.

Uniform cost search example (Find path from A to E)

- Expand A to B,C,D
- The path to B is the cheapest one with path cost 2.
- Expand B to E
 - Total path cost = $2 + 9 = 11$
- This might not be the optimal solution since the path AC as path cost 4 (less than 11)
- Expand C to E
 - Total path cost = $4 + 5 = 9$
- Path cost from A to D is 10 (greater than path cost, 9)
- Hence optimal path is ACE



Disadvantage

Does not care about the no of steps a path has but only about their cost. Hence it might get stuck in an infinite loop if it expands a node that has a zero cost action leading back to same state

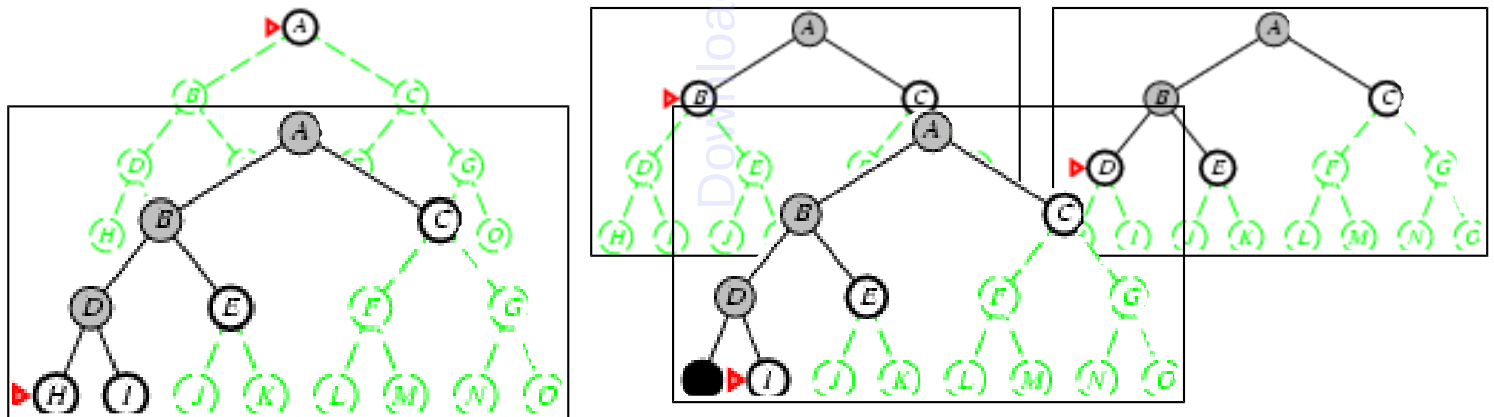
Properties of uniform cost search

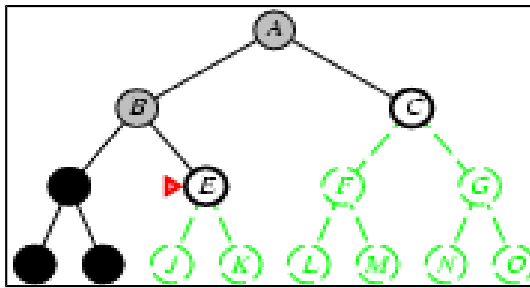
- **Completeness:** Complete if the cost of every step is greater than or equal to some small positive constant ϵ
- **Optimality:** Optimal if the cost of every step is greater than or equal to some small positive constant ϵ
- **Time complexity :** $O(b^{c^*/\epsilon})$ where c^* is cost of optimal path, ϵ is small positive constant
- **Space Complexity :** $O(b^{c^*/\epsilon})$

Depth-First Search (DFS)

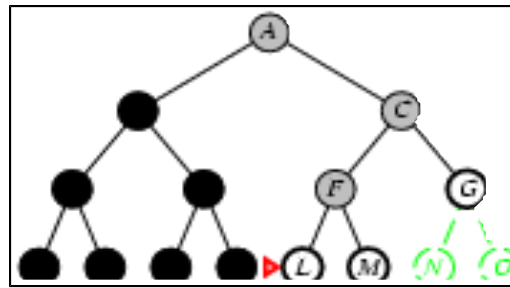
- DFS proceeds down a single branch of the tree at a time.
- It expands the root node, then the leftmost child of the root node, then the leftmost child of that node etc.
- Always expands a node at the deepest level of the tree
- Only when the search hits a dead end (a partial solution which can't be extended) does the search *backtrack and expand nodes at higher levels*.
- **Process: Use stack to keep track of nodes. (LIFO)**
 - Put the start node on the stack
 - While stack is not empty
 - Pop the stack
 - If the top of stack is the goal, stop
 - Otherwise push the nodes connected to the top of the stack on the stack (provided they are not already on the stack)

DFS example (find path from A to L)





DFS weakness



- We may get stuck going down an infinite branch that doesn't lead to a solution.
- As in given figure, suppose our solution is on right branch at depth level 1, but DFS search proceeds down left branch and continues its search upto 1000's of nodes. It might get stuck in that branch without solution.

Properties of depth-first search

- **Completeness:** Incomplete as it may get stuck down going down an infinite branch that doesn't leads to solution.
- **Optimality:** The first solution found by the DFS may not be shortest.
- **Space complexity:** b as branching factor and d as tree depth level, Space complexity = $O(b \cdot d)$
- **Time Complexity:** $O(b^d)$

Depth-Limited Search

- Breadth first has computational, especially, space problems.
- Depth first can run off down a very long (or infinite) path. Solution may be not be optimal.
- Depth limited search.
 - Perform depth first search but only to a pre-specified depth limit L.
 - No node on a path that is more than L steps from the initial state is placed on the Frontier.
 - We "truncate" the search by looking only at paths of length L or less.
- Now infinite length paths are not a problem.
- But will only find a solution if a solution of length $\leq L$ exists.

Properties of depth limit search

- **Completeness:** Incomplete as solution may be beyond specified depth level.
- **Optimality:** not optimal
- **Space complexity:** b as branching factor and l as tree depth level, Space complexity = $O(b \cdot l)$
- **Time Complexity:** $O(b^l)$

Iterative Deepening DFS

- Take the idea of depth limited search one step further.
- Starting at depth limit $L = 0$, we iteratively increase the depth limit, performing a depth limited search for each depth limit.
- Stop if no solution is found, or if the depth limited search failed without cutting off any nodes because of the depth limit.
- Iterative deepening search uses only linear space and not much more time than other uninformed algorithms
- Search is helpful only if the solution is at given depth level.

An example of Iterative deepening DFS (depth level 3)

Limit = 0



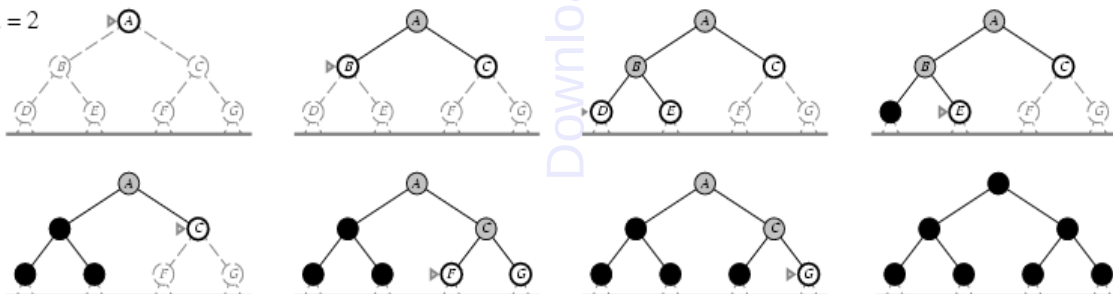
First Iteration Search at level $l=0$

Limit = 1



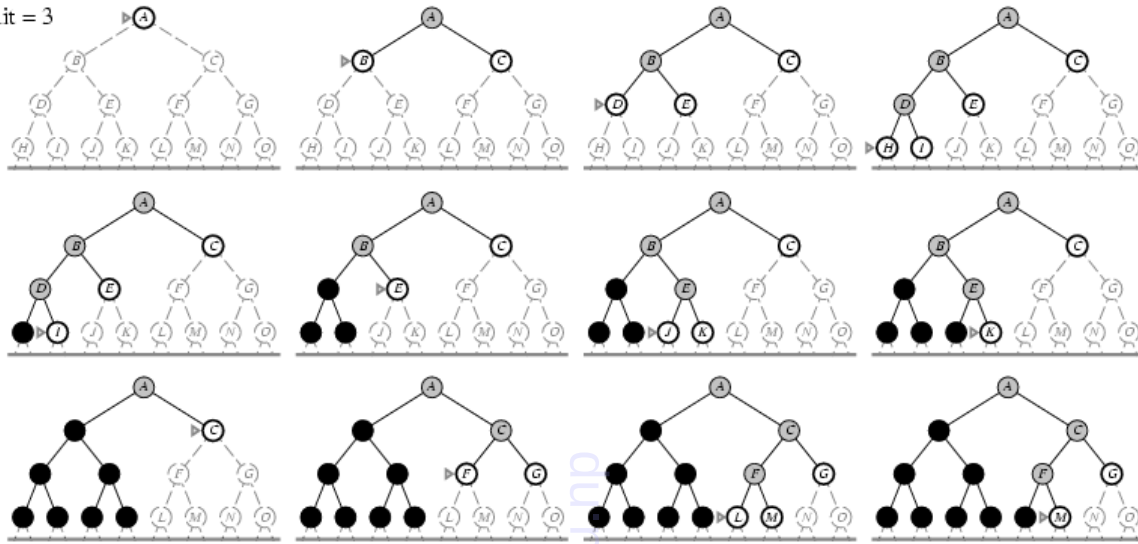
Second Iteration Search at level $l=1$

Limit = 2



Third Iteration Search at level $l=2$

Limit = 3



Fourth Iteration Search at level l=3

Informed Search (Heuristic Search)

- In uninformed search, we don't try to evaluate which of the nodes on the frontier are most promising. We never "look-ahead" to the goal
- Informed search have problem specific knowledge apart from problem definition.
- Use of Heuristic improves efficiency of search process.
- The idea is to develop a domain specific heuristic function $h(n)$.
- $h(n)$ guesses the cost of getting to the goal from node n .

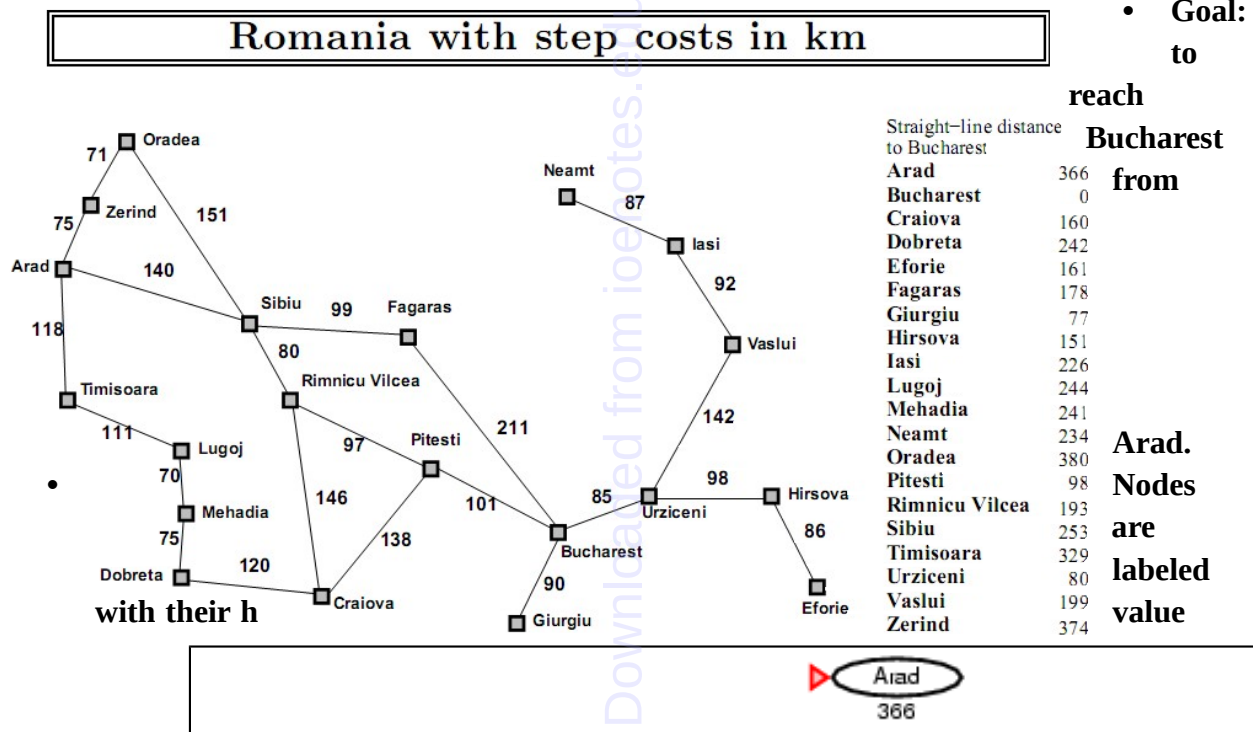
Best first Search

- A node is selected for expansion based on evaluation function $f(n)$
- Node with lowest evaluation function is expanded first.
- The evaluation function must represent some estimate of the cost of the path from state to the closest goal state.
- One important heuristic function is $h(n)$
- $h(n)$ is the estimate cost of the cheapest path from node to the goal
- Types
 - Greedy Best First Search
 - A* search

Greedy best-first search

- It tries to get as close as it can to the goal.
- It expands the node that appears to be closest to the goal
- It evaluates the node by using heuristic function only.
- Evaluation function $f(n) = h(n)$
 - (heuristic)= (estimate of cost from n to goal)
 - $h(n) = 0$ for goal state

example (find path from Arad to Bucharest)



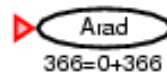
Evaluate cities connected to source and choose the one with lowest $h(\text{distance})$ value



- Complete? No – can get stuck in loops, e.g., $\text{Iasi} \neq \text{Neamt} \neq \text{Iasi} \neq \text{Neamt} \neq \dots$
- Time? $O(b^m)$, but a good heuristic can give dramatic improvement
- Space? $O(b^m)$ -- keeps all nodes in memory
- Optimal? No

- Greedy best first search analyses nodes with lowest cost of node n to reach goal. And it may get stuck in search of goal.
- Idea: avoid expanding paths that **are already expensive**
 Evaluation function $f(n) = g(n) + h(n)$
 $g(n)$ = cost so far to reach n
 $h(n)$ = estimated cost from n to goal
 $f(n)$ = estimated total cost of path through n to goal

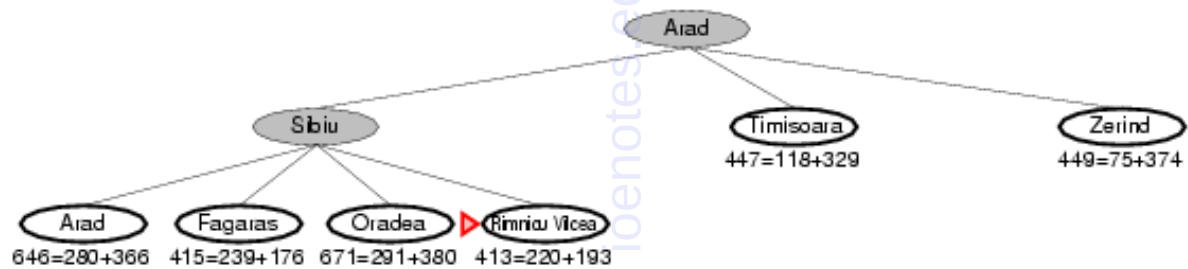
A* search example (find path from Arad to Bucharest)



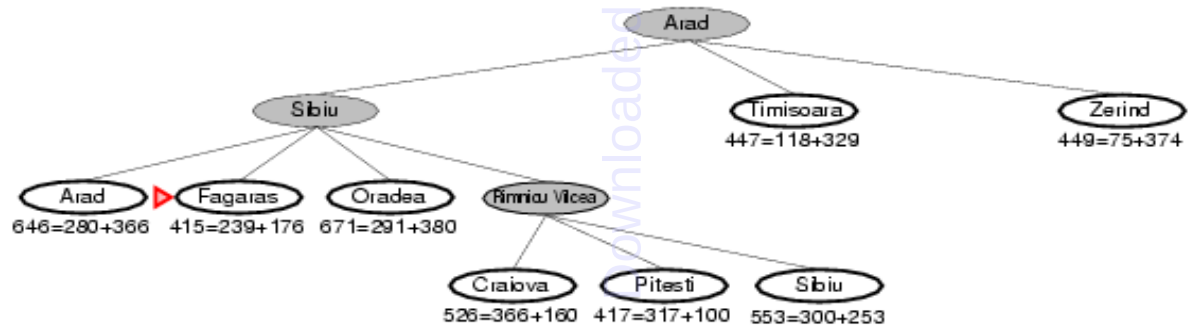
Start with source node



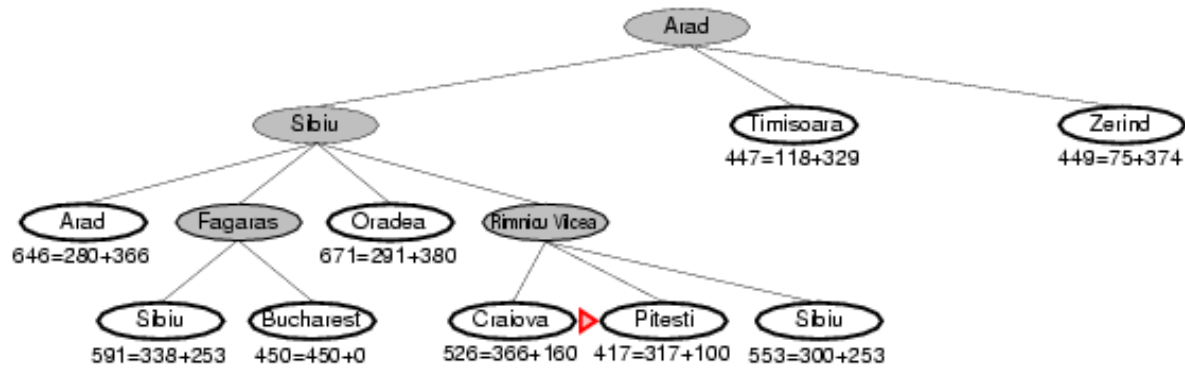
Evaluate nodes connected to source. Evaluate $f(n) = g(n) + h(n)$ for each node. Select node with lowest $f(n)$ value. *Sibiu node is chosen*



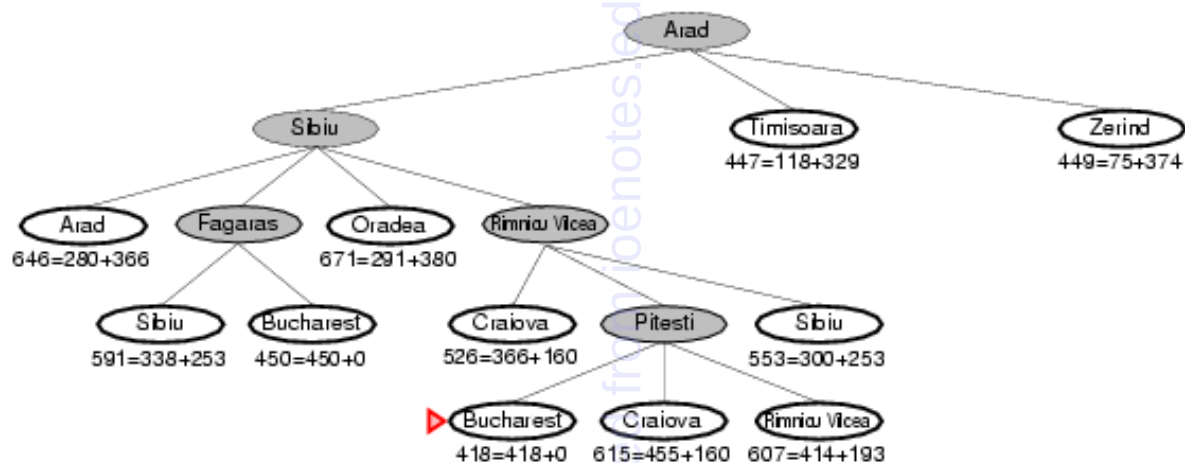
Rimniou vicea node has lowest $f(n)$ value, so this city is chosen



In all nodes expanded *Fagaras* has lowest $f(n)$ value, so this node is expanded next



Expanding *Fagaras* node reaches our destination Bucharest with $f(n)$ value 450. So all the nodes with $f(n)$ values expensive than 450 are avoided as evaluating them would result in cost of path expensive than our path already achieved. The nodes with $f(n)$ value lower than 450 are evaluated next.



Evaluating next node with *Petesti* with cost 417 (lower than 450 –Arad-Sibiu-Fagaras-Bucharest) we reach Bucharest with cost 418 which is lower than our earlier search result. All other nodes are expensive than our new result i.e. cost of 418. Hence our search is stopped here.

Admissible heuristics

- A heuristic $h(n)$ is admissible if for every node n , $h(n) \leq h^*(n)$, where $h^*(n)$ is the true cost to reach the goal state from n .
Example: $h_{SLD}(n)$ (never overestimates the actual road distance)
- An admissible heuristic never overestimates the cost to reach the goal, i.e., it is optimistic
- **Theorem:** If $h(n)$ is admissible, A* using GRAPH-SEARCH is complete
- Space Complexity = $O(b^d)$
- Time Complexity = $O(b^d)$

Consistent heuristics

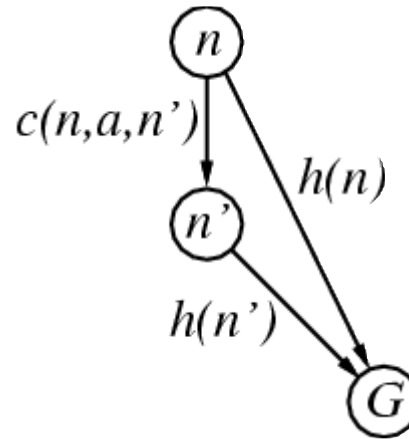
- A heuristic is consistent if for every node n , every successor n' of n generated by any action a ,

$$h(n) \leq c(n,a,n') + h(n')$$

- If h is consistent, we have

$$\begin{aligned} f(n') &= g(n') + h(n') \\ &= g(n) + c(n,a,n') + h(n') \\ &\geq g(n) + h(n) \\ &= f(n) \end{aligned}$$

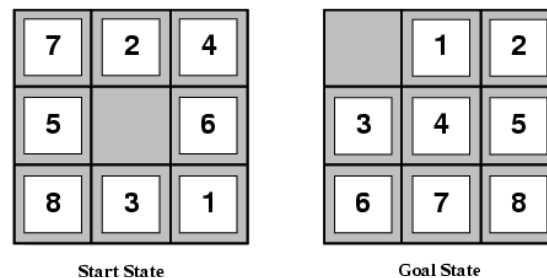
- i.e., $f(n)$ is non-decreasing along any path.
- Theorem:** If $h(n)$ is consistent, A* using GRAPH-SEARCH is optimal



Admissible heuristics example (8 puzzle)

E.g., for the 8-puzzle:

- $h_1(n)$ = number of misplaced tiles
- $h_2(n)$ = total Manhattan distance (i.e., the sum of the distances of the tiles from their goal positions)
- $h_1(S) = ?$ 8
- $h_2(S) = ?$ $3+1+2+2+2+3+3+2 = 18$



Local search and Optimization

- Local search
 - Keep track of single current state
 - Move only to neighboring states
- Advantages
 - Use very little memory

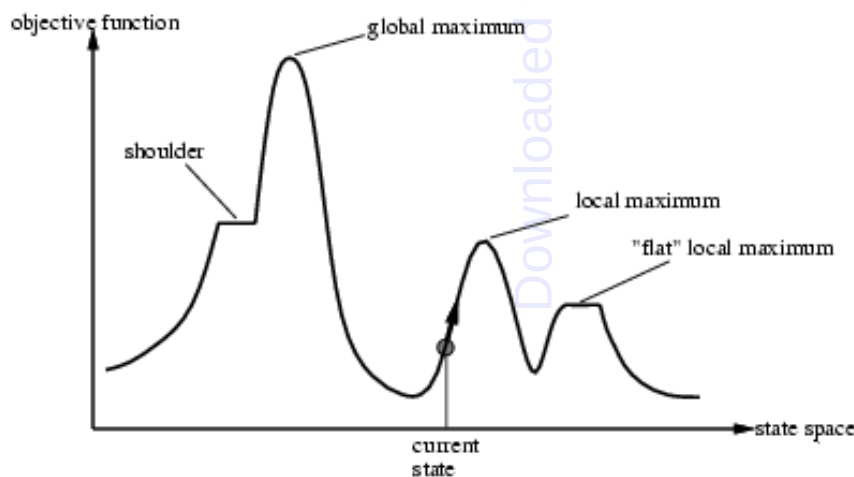
Iterative Improvement Algorithm

- Hill climbing search
- Simulated Annealing search

Hill climbing search

Hill climbing search is sometimes compared to climbing Mount Everest in thick fog with amnesia i.e. destined to reach top of Everest but due to thick fog (poor visibility) and forgetting what top of Everest is like, peak lower than top of Everest might be considered as top of Everest.

- Hill Climbing Search initiates a loop that continuously moves in the direction of increasing value
- Terminates when it reaches a “Peak”
- Doesn't maintain a search tree so the current node data structure needs only record the state and its objective function value.
- Hill climbing does not look ahead beyond the immediate neighbors of the current state.
- Hill climbing is also called greedy local search sometimes because it grabs a good neighbor state without thinking ahead about where to go next
- One move is selected and all other nodes are rejected and are never considered.
- Halts if there is no successor.
- **Problem: depending on initial state, can get stuck in local maxima**



Algorithm

- determine successors of current state
- choose successor of maximum goodness (break ties randomly)
- if goodness of best successor is less than current state's goodness, stop
- otherwise make best successor the current state and go to step 1

Drawback of Hill Climbing Search

This simple policy has three well-known drawbacks:

a. Local Maxima: a local maximum as opposed to global maximum.

b. Plateaus: An area of the search space where evaluation function is flat, thus requiring random walk. Search might be unable to find its way of plateau.

c. Ridge: Where there are steep slopes and the search direction is not towards the top but towards the side.

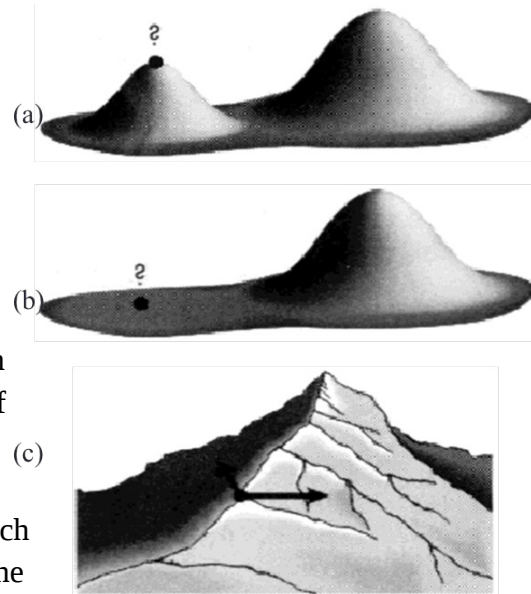


Figure 5.9 Local maxima, Plateaus and ridge situation for Hill Climbing

Solution

- Backtrack to some earlier node and try going to different direction (**for local maxima**)
- Make a big jump in some direction to try to get a new section of the search space (**for plateau**)
- Apply two or more rules such as bi-direction search before doing the test (**for ridge**)
 - Moving in several directions at once

Simulated Annealing Search

- Idea: escape local maxima by allowing some "bad" moves but gradually decrease their frequency
- Instead of restarting from a random point, we allow the search to take some downhill steps to try to escape local maxima.
- A random pick is made for the move
 - If it improves the situation, it is accepted straight away
 - If it worsens the situation, it is accepted with some probability less than 1 which decreases exponentially with the badness of the move i.e. for bad moves the probability is low and for comparatively less bad moves, it is higher.
- Probability of downward steps is controlled by **temperature parameter**.

- High temperature implies high chance of trying locally "bad" moves, allowing nondeterministic exploration.
- Low temperature makes search more deterministic (like hill-climbing).
- Temperature begins high and gradually decreases according to a predetermined annealing schedule.
- Initially we are willing to try out lots of possible paths, but over time we gradually settle in on the most promising path.
- If temperature is lowered slowly enough, an optimal solution will be found.
- In practice, this schedule is often too slow and we have to accept suboptimal solutions.

Applications of Simulated Annealing Search

- VLSI layout problem
- Factory scheduling
- Travelling salesman problem

Local beam search

- A path based algorithm
- Keeps track of k states rather than just one
- Starts with k randomly generated states
- At each iteration, all the successors of all k states are generated.
- If anyone is a goal state, stop, else select the k best successors from the complete list and repeat.

Adversarial Search

Competitive environments in which the agents goals are in conflict, give rise to **adversarial search**, often known as **games**.

In AI, games means deterministic, fully observable environments in which there are two agents whose actions must alternate and in which utility values at the end of the game are always equal and opposite.

- Eg. **If first player wins, the other player necessarily loses**

Opposition between the agent's utility functions make the situation adversarial.

Game

A game can be formally be defined as a kind of search problem with the following components

- Initial state

- A successor function
- A terminal test
- A utility function

Minimax Algorithm

- It is a **recursive algorithm** for choosing the next move in a n-player game, **usually a two player game**
- A value is associated with each position or state of the game
- The value is computed by means of a **position evaluation function** and **it indicates how good it would be for a player to reach the position.**
- The player then makes the move that maximizes the minimum value of the position from the opponents possible moves called maximizing player and other player minimize the maximum value of the position called minimizing player.

MiniMax Game Search

- It is a Depth-first search with limited depth.
- Use a static evaluation function for all leaf states.
- Assume the opponent will make the best move possible.

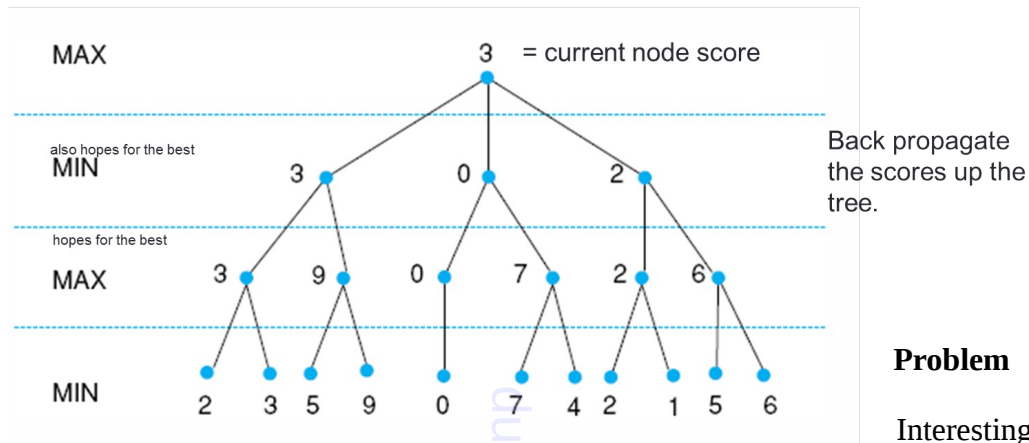
Algorithm

```

minimax(player,board)
    if(game over in current board position)
        return winner
    children = all legal moves for player from this board
    if(max's turn)
        return maximal score of calling minimax on all the children
    else (min's turn)
        return minimal score of calling minimax on all the children
  
```

example

Look 3 steps ahead.



- have too many states to expand to the leaves. Use heuristic $h(n)$ for each of these future positions.
- Number of nodes to expand is exponential in the depth of the tree and the branching factor.

Properties of minimax

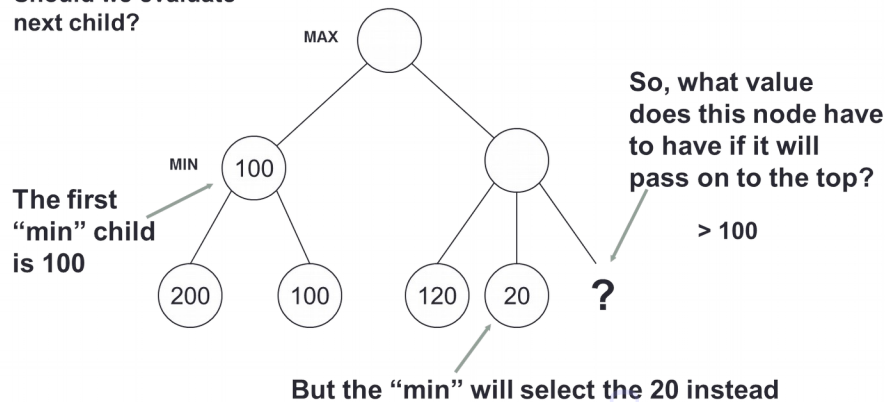
- Complete? Yes (if tree is finite)
- Optimal? Yes (against an optimal opponent)
- Time complexity? $O(b^m)$
- Space complexity? $O(bm)$ (depth-first exploration)
- For chess, $b \approx 35$, $m \approx 100$ for "reasonable" games
 □ exact solution completely infeasible

α - β pruning

- **Minimax search explores some parts of tree it doesn't have to**
- To avoid this, minimax algorithm is modified with two values **alpha** and **beta**, which represent the minimum score that the maximizing player is assured of and the maximum score that the minimizing player is assured respectively.
- Alpha-beta pruning algorithm **explores a branch only if there is possibility of producing superior alternatives.**
- It **reduces the time** required for the search and it must be restricted so that no time is to be wasted searching moves that are obviously bad for the current player.
- The exact implementation of alpha-beta keeps track of the best move for each side as it moves throughout the tree.
- It adapts minimax to consider the values of the nodes and whether exploration down a branch could possibly be fruitful.
- It proceeds from the idea **"If you have an idea that is surely bad, don't take the time to see how truly awful it is (Pat Winston)"**

Example

Should we evaluate next child?



Alpha Beta Procedure

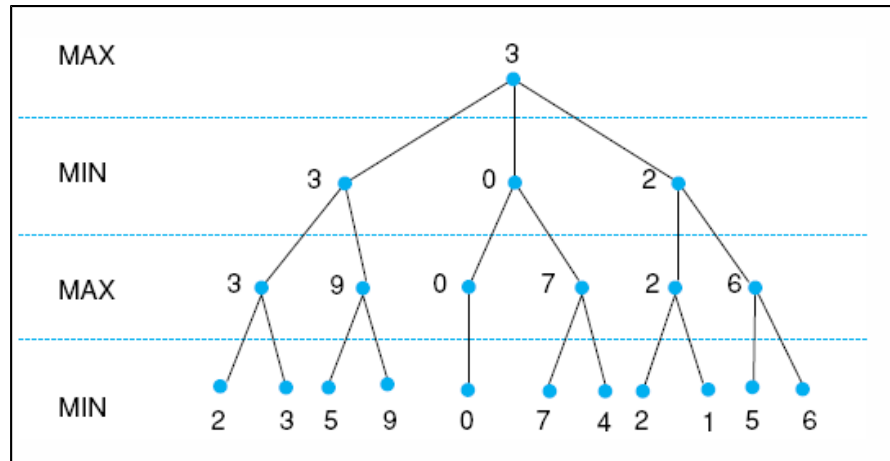
- At each non-leaf node, store two values— *alpha* and *beta*.
- Let *alpha* be the best (i.e., maximum) value found so far at a “max” node.
- Let *beta* be the best (i.e., minimum) value found so far at a “min” node.
- Initially assign $\alpha = -\infty$ and $\beta = \infty$ at the root.
- Note *alpha* is monotonically non-decreasing and *beta* is monotonically non-increasing as you travel *up* the tree.
- Given a node *n*, cut off the search below that node (i.e., generate no more children) if
 - *n* is a “max” node and $\alpha(n) \approx \beta(i)$ for some “min” ancestor *i* of *n*, or
 - *n* is a “min” node and $\beta(n) \geq \alpha(j)$ for some “max” ancestor *j* of *n*.

Algorithm

```

function alphabeta(node, depth,  $\alpha$ ,  $\beta$ , maximizingPlayer)
  if depth = 0 or node is a terminal node
    return the heuristic value of node
  if maximizingPlayer
    for each child of node
       $\alpha := \max(\alpha, \text{alphabeta}(\text{child}, \text{depth} - 1, \alpha, \beta, \text{FALSE}))$ 
      if  $\beta \leq \alpha$ 
        break (*  $\beta$  cut-off *)
    return  $\alpha$ 
  else
    for each child of node
       $\beta := \min(\beta, \text{alphabeta}(\text{child}, \text{depth} - 1, \alpha, \beta, \text{TRUE}))$ 
      if  $\beta \leq \alpha$ 
        break (*  $\alpha$  cut-off *)
    return  $\beta$ 
  
```

Example



Minimax without pruning

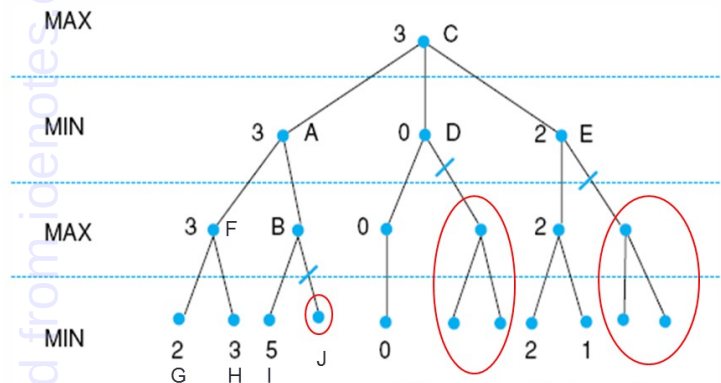
Alpha-Beta run

- Depth-first search
Visit C, A, F,
Visit G, heuristics evaluates to 2
Visit H, heuristics evaluates to 3
- Back up {2,3} to F. $\max(F)=3$
- Back up to A. $\beta(A)=3$. Temporary $\min(A)$ is 3.
3 is the ceiling for node A's score.
- Visit B according to depth-first order.
- Visit I. Evaluates to 5.
- $\text{Max}(B) \geq 5$. $\alpha(B)=5$.
It does not matter what the value of J is, $\min(A)=3$. β -prune J.

Alpha-beta pruning improves search efficiency of minimax without sacrificing accuracy.

Effectiveness

- The effectiveness depends on the order in which children are visited.
- In the best case, the effective branching factor will be reduced from b to $\sqrt{b}$.
- In an average case (random values of leaves) the branching factor is reduced to $b/\log b$.



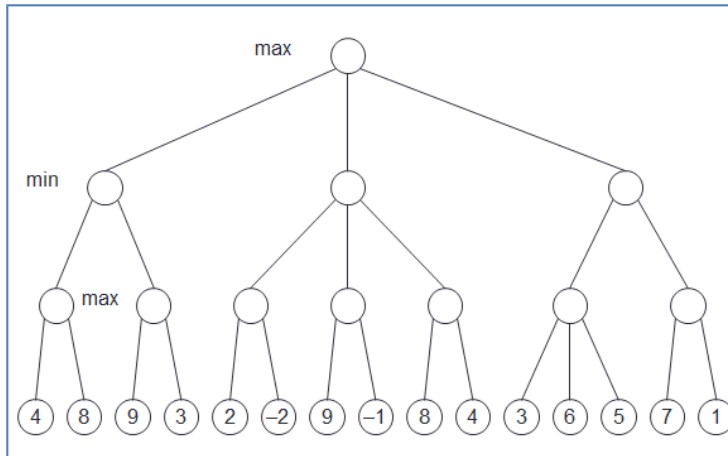
Unevaluated nodes

- A has $\beta = 3$ (A will be no larger than 3)
- B is β pruned, since $5 > 3$
- C has $\alpha = 3$ (C will be no smaller than 3)
- D is α pruned, since $0 < 3$
- E is α pruned, since $2 < 3$
- C is 3

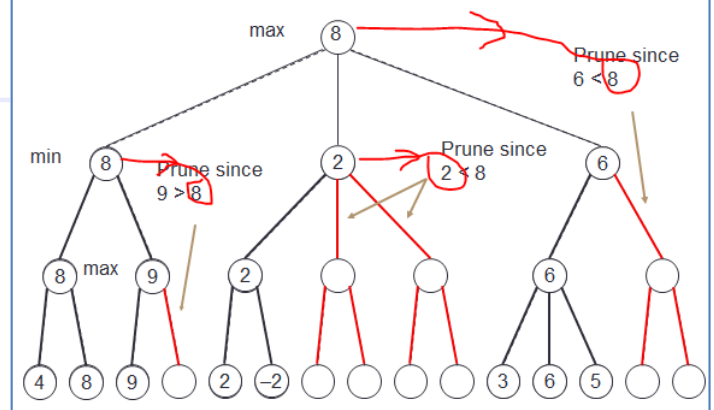
Properties of α - β

- Pruning does not affect final result
- Good move ordering improves effectiveness of pruning
- With "perfect ordering," time complexity = $O(b^{m/2})$
□ doubles depth of search
- A simple example of the value of reasoning about which computations are relevant

Example



Alpha Beta Run



Chapter- 4

Knowledge Representation

- The objective of knowledge representation is to express the knowledge about the world in a computer-tractable form
- Key aspects of knowledge representation languages are:
 - i. Syntax: describes how sentences are formed in the language.
 - ii. Semantics: describes the meaning of sentences, what is it the sentence refers to in the real world.
 - iii. Computational aspect: describes how sentences and objects are manipulated in concordance with semantically conventions.
- A *formal language* is required to represent knowledge in a computer tractable form and *reasoning* processes are required to manipulate this knowledge to deduce non-obvious facts.

Knowledge Representation using Logic

- Logic is defined as a formal language for expressing knowledge and ways of reasoning.
- Logic makes statements about the world which are true (or false).
- Compared to natural languages (expressive but context sensitive) and programming languages (good for concrete data structures but not expressive) logic combines the advantages of natural languages and formal languages.
- Logic is:
 - concise
 - unambiguous
 - context insensitive
 - expressive
 - effective for inferences
- A logic is defined by the following:
 1. **Syntax** - describes the possible configurations that constitute sentences.
 2. **Semantics** - determines what facts in the world the sentences refer to i.e. the interpretation. Each sentence makes a claim about the world.
 3. **Proof theory** - set of rules for generating new sentences that are necessarily true given that the old sentences are true. The relationship between sentences is

called **entailment**. The semantics link these sentences (representation) to facts of the world. The proof can be used to determine new facts which follow from the old.

4. **A set of sentences** – A sentence is constructed from a set of primitives according to syntax rules.
 5. **A set of interpretations** – An interpretation gives a semantic to primitives. It associates primitives with values.
 - **The valuation (meaning) function** – Assigns a value (typically the truth value) to a given sentence under some interpretation. $\text{sentence} \times \text{interpretation} \rightarrow \{\text{True}, \text{False}\}$
- Knowledge representation using logic shows:
 - i. How logic can be used to form representations of the world?
 - ii. How a process of inference can be used to derive new representations about the world?
 - iii. How these can be used by an intelligent agent to deduce what to do.
 - **Types of logic:** Different types of logics are: Propositional logic and First-order logic (First –order Predicate Logic).

Propositional logic

- A propositional logic is a declarative sentence which can be either true or false but not both or either.
- Propositional logic is a mathematical model that allows us to reason about the truth or falsehood of logical expression.
- In propositional logic, there are atomic sentences and compound sentences built up from atomic sentences using logical connectives.

A set of connectives:



AND



OR



NOT

$\Rightarrow$ implies

$\Leftrightarrow$ mutual implication

Sentences in the propositional logic:

- i. **Atomic sentences:** – Constructed from constants and propositional symbols – True, False are (atomic) sentences, Light in the room is on, It rains outside are (atomic) sentences.
- ii. **Composite sentences:** – Constructed from valid sentences via connectives eg: $(A \wedge B) (A \vee B) (A \Rightarrow B) (A \Leftrightarrow B) (A \vee B) \wedge (A \vee \neg B)$

• A sentence (well formed formula) is defined as follows:

- A symbol is a sentence
- If S is a sentence, then $\neg S$ is a sentence
- If S is a sentence, then (S) is a sentence
- If S and T are sentences, then $(S \neg T)$, $(S \vee T)$, $(S \wedge T)$, and $(S \leftrightarrow T)$ are sentences

A sentence results from a finite number of applications of the above rules

Truth Table in Propositional Logic

A	B	$\neg A$	$A \vee B$	$A \wedge B$	$A \Rightarrow B$	$A \oplus B$
T	T	F	T	T	T	T
T	F	F	T	F	F	F
F	T	T	T	F	T	F
F	F	T	F	F	T	T

Find Truth Table of following :

- $A \Rightarrow (B \Rightarrow A)$
- $(A \Rightarrow B) \Rightarrow A$
- $A \Rightarrow ((B \wedge C) \vee \neg A)$
- $((A \wedge B) \rightarrow C) \Leftrightarrow (A \rightarrow (B \rightarrow C))$.

Tautology: Tautology is notation in formal logic which is always true (valid).

Contradiction (Unsatisfiable): notation in formal logic which is always false.

Satisfiable: If at least one sentence is true in the set.

Important Logical Equivalences: First List

The following logical equivalences apply to *any* statements; the p's, q's and r' s can stand for atomic statements or compound statements.

$$\sim(\sim p) \equiv p$$

the Double Negative Law

$$\sim(p \vee q) \equiv (\sim p) \wedge (\sim q)$$

De Morgan's Laws

$$\sim(p \wedge q) \equiv (\sim p) \vee (\sim q)$$

$$p \wedge (q \vee r) \equiv (p \wedge q) \vee (p \wedge r)$$

the Distributive Laws

$$p \vee (q \wedge r) \equiv (p \vee q) \wedge (p \vee r)$$

Inference rules in propositional logic:

Rule of inference	Tautology	Name
$\begin{array}{l} p \rightarrow q \\ p \\ \hline \therefore q \end{array}$	$[p \wedge (p \rightarrow q)] \rightarrow q$	Modus ponens
$\begin{array}{l} \neg q \\ p \rightarrow q \\ \hline \therefore \neg p \end{array}$	$[\neg q \wedge (p \rightarrow q)] \rightarrow \neg p$	Modus tollens
$\begin{array}{l} p \rightarrow q \\ q \rightarrow r \\ \hline \therefore p \rightarrow r \end{array}$	$[(p \rightarrow q) \wedge (q \rightarrow r)] \rightarrow (p \rightarrow r)$	Hypothetical syllogism
$\begin{array}{l} p \vee q \\ \neg p \\ \hline \therefore q \end{array}$	$((p \vee q) \wedge \neg p) \rightarrow q$	Disjunctive syllogism
$\begin{array}{l} p \\ \hline \therefore p \vee q \end{array}$	$p \rightarrow (p \vee q)$	Addition
$\begin{array}{l} p \wedge q \\ \hline \therefore p \end{array}$	$(p \wedge q) \rightarrow p$	Simplification
$\begin{array}{l} p \\ q \\ \hline \therefore p \wedge q \end{array}$	$((p) \wedge (q)) \rightarrow (p \wedge q)$	Conjunction
$\begin{array}{l} p \vee q \\ \neg p \vee r \\ \hline \therefore q \vee r \end{array}$	$[(p \vee q) \wedge (\neg p \vee r)] \rightarrow (p \vee r)$	Resolution

Predicate Logic

- Predicate logic is an extension of propositional logic with more expressive power.
- In propositional logic, each possible atomic fact requires a separate unique propositional symbol.
- If there are n people and m locations, representing the fact that some person moved from one location to another requires (nm^2) separate symbols.
- Predicate logic includes a richer ontology: -objects (terms) -properties (unary predicates on terms) -relations (n -ary predicates on terms) -functions (mappings from terms to other terms)
- Allows more flexible and compact representation of knowledge: $\text{Move}(x, y, z)$ for person x moved from location y to z .
- Predicate Symbols are used to denote a property of objects or a relation between objects

Syntax for First-Order Logic

Sentence $\rightarrow$ AtomicSentence
 | Sentence Connective Sentence
 | Quantifier Variable Sentence
 | $\neg$ Sentence
 | (Sentence)
AtomicSentence $\rightarrow$ Predicate(Term, Term, ...)
 | Term=Term
Term $\rightarrow$ Function(Term,Term,...)
 | Constant
 | Variable
Connective $\rightarrow \vee \mid \wedge \mid \Rightarrow \mid \Leftrightarrow$
Quantifier $\rightarrow \exists \mid \forall$
Constant $\rightarrow A \mid \text{John} \mid \text{Carl}$
Variable $\rightarrow x \mid y \mid z \mid \dots$
Predicate $\rightarrow \text{Brother} \mid \text{Owns} \mid \dots$
Function $\rightarrow \text{father-of} \mid \text{plus} \mid \dots$

First-Order Logic: Terms and Predicates

- Objects are represented by **terms**:
 - **Constants**: Block1, John
 - **Function symbols**: father-of, successor, plus
An n -ary function maps a tuple of n terms to another term: father-of(John), successor(0), plus(plus(1,1),2)
- Terms are simply names for objects. Logical functions are not procedural as in programming languages. They do not need to be defined, and do not really return a value. Allows for the representation of an infinite number of terms.
- Propositions are represented by a **predicate** applied to a tuple of terms. A predicate represents a property of or relation between terms that can be true or false:
Brother(John, Fred), Left-of(Square1, Square2)
GreaterThan(plus(1,1), plus(0,1))
- In a given interpretation, an n -ary predicate can be defined as a function from tuples of n terms to $\{\text{True}, \text{False}\}$ or equivalently, a set of tuples that satisfy the predicate:
 $\{ \langle \text{John}, \text{Fred} \rangle, \langle \text{John}, \text{Tom} \rangle, \langle \text{Bill}, \text{Roger} \rangle, \dots \}$

Sentences in First-Order Logic

- An atomic sentence is simply a predicate applied to a set of terms.

Owens(John,Car1)
Sold(John,Car1,Fred)

Semantics is True or False depending on the interpretation, i.e. is the predicate true of these arguments.

- The standard propositional connectives ($\vee$ $\neg$ $\wedge$ $\Rightarrow$ $\Leftrightarrow$) can be used to construct complex sentences:

Owens(John,Car1) $\vee$ Owens(Fred, Car1)
Sold(John,Car1,Fred) $\Rightarrow$ $\neg$ Owens(John, Car1)

Semantics same as in propositional logic.

Variable Scope

- The **scope** of a variable is the sentence to which the quantifier syntactically applies.
- As in a block structured programming language, a variable in a logical expression refers to the closest quantifier within whose scope it appears.

$\exists x (Cat(x) \wedge \forall x(Black(x)))$

The x in Black(x) is universally quantified

Says cats exist and everything is black

- In a **well-formed formula (wff)** all variables should be properly introduced:

$\exists xP(y)$ not well-formed

- A **ground** expression contains no variables.

Quantifiers

- Allows statements about entire collections of objects rather than having to enumerate the objects by name.

- Universal quantifier: $\forall x$

Asserts that a sentence is true for all values of variable x

$\forall x Loves(x, FOPC)$

$\forall x Whale(x) \Rightarrow Mammal(x)$

$\forall x Grackles(x) \Rightarrow Black(x)$

$\forall x (\forall y Dog(y) \Rightarrow Loves(x,y)) \Rightarrow (\forall z Cat(z) \Rightarrow Hates(x,z))$

- Existential quantifier: $\exists$

Asserts that a sentence is true for at least one value of a variable x

$\exists x Loves(x, FOPC)$

$\exists x (Cat(x) \wedge Color(x,Black) \wedge Owns(Mary,x))$

$\exists x (\forall y Dog(y) \Rightarrow Loves(x,y)) \wedge (\forall z Cat(z) \Rightarrow Hates(x,z))$

Relation Between Quantifiers

- Universal and existential quantification are logically related to each other:

$\forall x \neg Love(x, Saddam) \Leftrightarrow \neg \exists x Loves(x, Saddam)$

$\forall x Love(x, Princess-Di) \Leftrightarrow \neg \exists x \neg Loves(x, Princess-Di)$

- General Identities

$\neg \forall x \neg P \Leftrightarrow \neg \exists x P$

$\neg \forall x P \Leftrightarrow \exists x \neg P$

$\neg \forall x P \Leftrightarrow \neg \exists x \neg P$

$\neg \exists x P \Leftrightarrow \neg \forall x \neg P$

$\neg \forall x P(x) \wedge Q(x) \Leftrightarrow \forall x P(x) \wedge \forall x Q(x)$

$\neg \exists x P(x) \vee Q(x) \Leftrightarrow \exists x P(x) \vee \exists x Q(x)$

**** Refer Class Note for Fopl conversation, Inference rules in propositional logic, CNF conversion process and Proof by Resolution (RRS)**

Chapter-4

Rule-Based Systems

- Rule-based systems (RBS) provide automatic problem solving tools for capturing the human expertise and decision making.
- RBS are means for codifying the problem solving of human experts.
- Experts typically express most of their problem solving techniques in terms of antecedent-consequent rules.
- The main properties of rule-based systems are:
 - They incorporate practical human knowledge in if-then rules;
 - Their skill increases proportionally to the enlargement of their knowledge;
 - They can solve a wide range of potentially complex problems by selecting relevant rules and then combining the results;
 - They adaptively determine the best sequence of rules to examine;
 - They explain their conclusions by retracting their lines of reasoning

Forward chaining

- Forward chaining is one of the two main methods of reasoning when using [inference rules](#)
- Described [logically](#) as repeated application of [modus ponens](#).
- Forward chaining is a popular implementation strategy for [expert systems](#), [business](#) and [production rule systems](#).
- Forward chaining starts with the available [data](#) and uses inference rules to extract more data until a [goal](#) is reached.
- An [inference engine](#) using forward chaining searches the inference rules until it finds one where the [antecedent](#) (If clause) is known to be true. When such a rule is found, the engine can conclude, or infer, the [consequent](#) (Then clause), resulting in the addition of new [information](#) to its data.
- Inference engines will [iterate](#) through this process until a goal is reached.
- Because the data determines which rules are selected and used, this method is called [data-driven](#).
- The forward chaining approach is often employed by [expert systems](#), such as [CLIPS](#).
- One of the advantages of forward-chaining over backward-chaining is that the reception of new data can trigger new inferences, which makes the engine better suited to dynamic situations in which conditions are likely to change.

Example

The [rule base](#) contains the following four rules:

- If X croaks and eats flies - Then X is a frog
- If X chirps and sings - Then X is a canary
- If X is a frog - Then X is green
- If X is a canary - Then X is yellow

Goal: conclude the color of a pet named Fritz, given that he croaks and eats flies,

Inference Mechanism:

- This rule base would be searched and the first rule would be selected, because its antecedent (If Fritz croaks and eats flies) matches our data. Now the consequents (Then X is a frog) is added to the data.
- The rule base is again searched and this time the third rule is selected, because its antecedent (If Fritz is a frog) matches our data that was just confirmed. Now the new consequent (Then Fritz is green) is added to our data.
- Nothing more can be inferred from this information, but we have now accomplished our goal of determining the color of Fritz.

Backward chaining

- Backward chaining (or backward reasoning) is an [inference](#) method that can be described as working backward from the goal(s).
- It is used in [automated theorem provers](#), [proof assistants](#) and other [artificial intelligence](#) applications.
- In [game theory](#), its application to [subgames](#) in order to find a solution to the game is called [backward induction](#).
- In chess, it is called [retrograde analysis](#), and it is used to generate tablebases for [chess endgames](#) for [computer chess](#).
- Backward chaining is implemented in [logic programming](#) by [SLD resolution](#)
- Rules are based on the [modus ponens](#) inference rule.
- It is one of the two most commonly used methods of [reasoning](#) with [inference rules](#) and [logical implications](#).
- Backward chaining systems usually employ a [depth-first search](#) strategy.
- Backward chaining starts with a list of [goals](#) (or a [hypothesis](#)) and works backwards from the [consequent](#) to the [antecedent](#) to see if there is [data](#) available that will support any of these consequents.
- An [inference engine](#) using backward chaining would search the [inference](#) rules until it finds one which has a consequent (Then clause) that matches a desired goal.
- If the antecedent (If clause) of that rule is not known to be true, then it is added to the list of goals.
- Because the list of goals determines which rules are selected and used, this method is called [goal-driven](#), in contrast to [data-driven forward-chaining](#) inference. The backward chaining approach is often employed by [expert systems](#).

Example

The [rule base](#) contains the following four rules:

- If X croaks and eats flies - Then X is a frog
- If X chirps and sings - Then X is a canary
- If X is a frog - Then X is green
- If X is a canary - Then X is yellow

Goal: conclude the color of a pet named Fritz, given that he croaks and eats flies,

Inference Mechanism:

- This rule base would be searched and the third and fourth rules would be selected, because their consequents (Then Fritz is green, Then Fritz is yellow) match the goal (to determine Fritz's color).
- It is not yet known that Fritz is a frog, so both the antecedents (If Fritz is a frog, If Fritz is a canary) are added to the goal list.
- The rule base is again searched and this time the first two rules are selected, because their consequents (Then X is a frog, Then X is a canary) match the new goals that were just added to the list.
- The antecedent (If Fritz croaks and eats flies) is known to be true and therefore it can be concluded that Fritz is a frog, and not a canary.
- The goal of determining Fritz's color is now achieved (Fritz is green if he is a frog, and yellow if he is a canary, but he is a frog since he croaks and eats flies; therefore, Fritz is green).

Statistical Reasoning

Probability & Bayes Theorem

- An important goal for many problems solving systems is to collect evidence as the system goes along and to modify its behavior on the basis of evidence.
- To model this behavior we need a statistical theory of evidence.
- Bayesian statistics is such a theory. The fundamental notion of Bayesian statistics is that of condition probability.
- Read the expression as the probability of Hypothesis H given that we have observed evidence E.
- To compute this, we need to take into account the prior probability of H (the probability that we would assign to H if we had no evidence) & the extent to which E provides evidence of H. To do this we need to define a universe that contains an exhaustive, mutually exclusive set of H_i , among which we are trying to discriminate
- $P(H_i|E)$ = The probability that hypothesis H_i is true given evidence E, $P(E|H_i)$ (The probability that we will observe evidence E given that hypothesis H_i is true.)
- $P(H_i)$ = The apriori probability that hypothesis H_i is true in the absence of any specific evidence. These probabilities are called prior probabilities.
- Bayes theorem states then that
$$P(H_i|E) = P(E|H_i) * P(H_i) / P(E)$$

Bayesian Networks (Bayes N/W, Belief N/W, Directed Acyclic Graph)

- It is a probabilistic graphical model that represents a set of random variables and their conditional dependencies via a directed acyclic graph.
- Nodes represent the random variables and the edges represent conditional dependencies.

- Each node is associated with a probability function.
- Disadvantage:
 - i. Loop can occur sometimes
 - ii. The number of paths to explore exponentially with each node.

Eg: Refer Class note

Entailment (Logical consequence)

- Entailment is the relationship between current sentences and new sentences derived from the current sentences.
- Entailment is one of the most fundamental [concepts](#) in [logic](#).
- It is the relationship between [statements](#) that holds true when one logically "follows from" one or more others.
- A [valid](#) logical [argument](#) is one in which the [conclusions](#) follow from its [premises](#), and its conclusions are consequences of its premises.
- Entailment reflects the relation of one fact in the world following from the others.
- Knowledge base (KB) entails sentence if and only if it is true in all worlds where KB is true.
- **Sound and complete inference:** If a knowledgebase is true then any sentences α derived from knowledgebase by a sound inference is also true. Completeness is a condition if it can derive any sentences that are entailed.
- An inference algorithm that derives only entailed sentences is called sound or truth preserving.

Horn Clause

- Horn clause is a logical formula of a particular rule-like form which gives it useful properties for use in [logic programming](#).
- A Horn clause is a [clause](#) (a [disjunction](#) of [literals](#)) with at most one positive literal.
- A Horn clause with exactly one positive literal is a **definite clause**.
Eg; $\neg p \vee \neg q \vee \dots \vee \neg t \vee u$ (assume that, if p and q and ... and t all holds, then also u holds)
- A definite clause with no negative literals is sometimes called a **fact**.
Eg: u (assume that u holds).
- Horn clause without a positive literal is sometimes called a **goal clause** i.e. the empty clause consisting of no literals is a goal clause.
Eg: $\neg p \vee \neg q \vee \dots \vee \neg t$ (show that p and q and ... and t all holds)

Well-Formed Formula (WFF)

Rules for constructing Wffs

A predicate name followed by a list of variables such as $P(x, y)$, where P is a predicate name,

and x and y are variables, is called an atomic formula.

Wffs are constructed using the following rules:

1. True and False are wffs.
2. Each propositional constant (i.e. specific proposition), and each propositional variable (i.e. a variable representing propositions) are wffs.
3. Each atomic formula (i.e. a specific predicate with variables) is a wff.
4. If A , B , and C are wffs, then so are $\neg A$, $(A \wedge B)$, $(A \vee B)$, $(A \rightarrow B)$, and $(A \leftrightarrow B)$.
5. If x is a variable (representing objects of the universe of discourse), and A is a wff, then so are $\forall x A$ and $\exists x A$.

Chapter-5: Structured Knowledge Representation

- Knowledge Representation is an area of AI whose fundamental goal is to represent knowledge in manner that facilitates inference i.e. drawing conclusion for knowledge.
- It analyzes how to think formally, how to use symbol to represent a domain of discourse along with function that allow inference about the objects.
- Knowledge representation helps to address the problem like:
 - . How do we represent fact about the world?
 - . How do we reason about them?
 - . What representations are appropriate for dealing with the real world?
 - . How to express knowledge in computer understandable form so that reasoning agent can perform well?

Knowledge Mappings

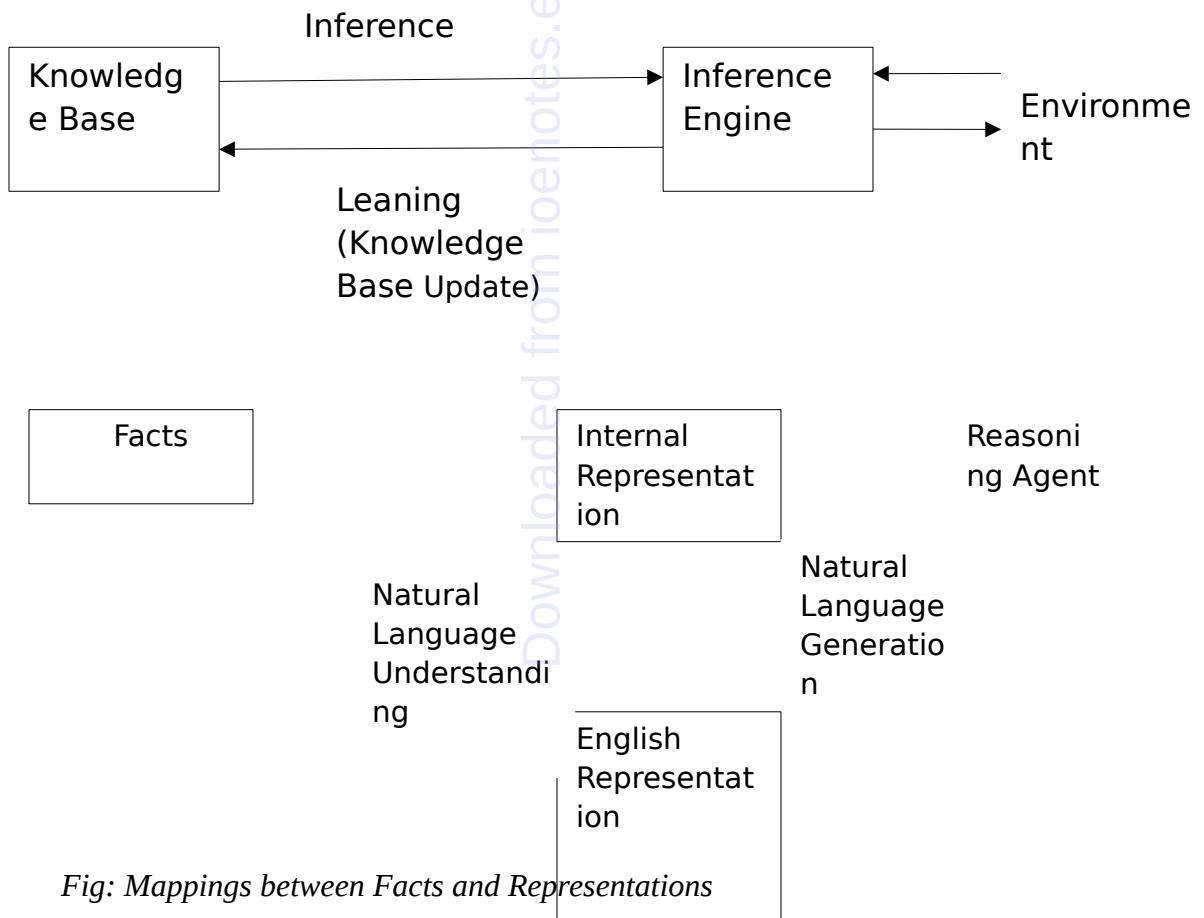


Fig: Mappings between Facts and Representations

Approaches to Knowledge Representation

A good system for knowledge representation should have

i. Representational Adequacy:

Ability to represent all kind of knowledge that are needed in the domain.

ii. Inferential Adequacy:

Ability to manipulate the representational structure in such a way as to derive new structures corresponding to new knowledge inferred from old.

iii. Inferential Efficiency:

Ability to incorporate into the knowledge structure additional information that can be used to focus the attention of the inference mechanism in the most efficient directions.

iv.

v. Acquisitional Efficiency:

Ability to acquire new information easily.

Knowledge Types:

a. Simple relational knowledge:

The simplest way to represent declarative facts is as a set of relations of the same sort used in database system.

b. Inheritable Knowledge

Structure must be designed to corresponding to the inference mechanism that are desired.

c. Inferential Knowledge

Represents knowledge as formal logic. Based on reasoning from facts or from other inferential knowledge. Useless unless there is also as inference procedure that can exploit it.

d. Procedural Knowledge (Imperative Knowledge)

Knowledge exercised in the performance of some task and processed by an intelligent agent.

Issues in Knowledge Representation

- Are any attributes of objects so basic that they have been occurred in almost every problem domain?
- Are there any important relationships that exist among attributes of objects?
- At what level should knowledge be represented?
- How should sets of objects be represented?
- How can relevant part be accessed when they are needed?

Knowledge can be viewed as different level.

- i. Knowledge Level:** The most abstract level that describe agent by saying what it knows. Eg: An intelligent taxi might know that the Bagmati bridge connects Thapathali with Kupondole.

- ii. **Logical Level:** The level at which the knowledge is encoded into formal sentences.
Eg: Links (Bagmati Bridge, Thapathali, Kupondole)
- iii. **Implementation Level:** Physical Representation of the sentences in the logical level.
Eg: Objects, Dam, Piller etc.

Knowledge Model

- A model is a world in which a sentence is true under a particular interpretation.
- There can be several models at once that have the same interpretation.

Types:

1. First Order Predicate Logic

- This consists of objects, predicates on objects, connectives and quantifiers
- Predicates are the relations between objects or properties of the objects.
- Connectives and quantifiers allows for universal sentences.
- Relation between objects can be true or false.

2. Procedural Representation Model

- This model of knowledge representation encodes facts along with the sequence of operations for manipulation and processing of the facts.
- Expert systems are based on this model.
- It works best when expert follows set of procedures for problem solving. Eg: A medical diagnosis system.

3. Relational Representation Model

- Collections of knowledge are stored in tabular form.
- Mostly used in commercial databases, relational databases.
- The information is manipulated with relational calculus using a language such as SQL, Oracle etc.
- This is flexible way to store information but not good for storing complex relationships.
- Problem arises when more than one subject area is attempted.
- A new knowledgebase from scratch has to be built for each area of expertise.

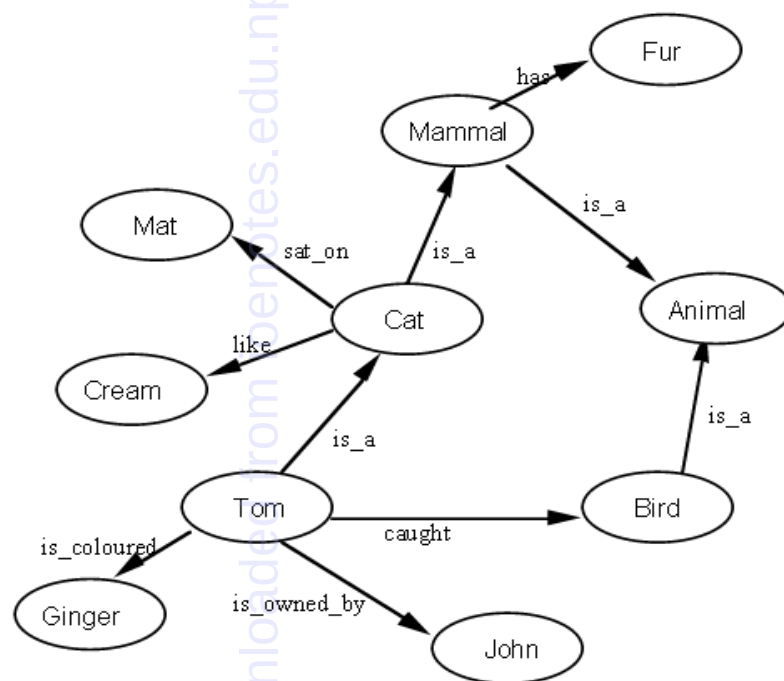
4. Hierarchical Representation Model

- Based on inherited knowledge and relationship and shared attributes between objects.
-

5. Semantic Nets

- Semantic network is an alternative to predicate logic as a form of knowledge representation.
- The structure of a semantic net is shown graphically in terms of nodes and the arcs connecting them.
 - . Nodes are sometimes referred to as objects.
 - . Arcs represent the links or edges.
 - . The links are used to express relationships.
- Two types of commonly used links are i. IS-A, and ii.A-KIND-OF
- IS-A means "is an instance of" and refers to a specific member of a class.

- The link A-KIND-OF is used here to relate one class to another .
- Nodes are to represent physical objects, concepts, or situation.
- The idea is that we can store our knowledge in the form of a graph, with nodes representing objects in the world, and arcs representing relationships between those objects.
- Semantic network is a declarative graphic representation that can be used to represent knowledge and support automated systems for reasoning about the knowledge
- For example, the following:



- Semantic Network is intended to represent the data:
- Tom is a cat.
- Tom caught a bird.
- Tom is owned by John.
- Tom is ginger in colour.
- Cats like cream.
- The cat sat on the mat.
- A cat is a mammal.
- A bird is an animal.
- All mammals are animals.
- Mammals have fur.
- It is argued that this form of representation is closer to the way human's structure knowledge.

Frames (slot and filler structure)

- Frame is a type of schema used in many [AI](#) applications including vision and [natural language processing](#) that provides a convenient structure for representing.
- Frame is similar to a record structure and corresponding to the fields and values are slots and slot fillers.
- Basically it is a group of slots and fillers that defines a stereotypical object.
- A single frame is not much useful. Frame systems usually have collection of frames connected to each other.
- Value of an attribute of one frame may be another frame.
- Frames are also useful for representing commonsense knowledge.
- While semantic nets are basically a two-dimensional representation of knowledge, frames add a third dimension by allowing nodes to have structures.
- By using frames in the filler slots and inheritance, very powerful knowledge representation systems can be built.
- Frame-based expert systems are very useful for representing causal knowledge because their information is organized by cause and effect.
- Frames are generally designed to represent either generic or specific knowledge.
- A frame for a book is given below.

Slots	Fillers
publisher	Thomson
title	Expert Systems
author	Giarratano
edition	Third
year	1998
pages	600

Conceptual Dependency (CD) (Strong slot and filler structure)

- Conceptual Dependency originally developed to represent knowledge acquired from natural language input.
- The goals of this theory are:
 - To help in the drawing of inference from sentences.
 - To be independent of the words used in the original input.
- For any 2 (or more) sentences that are identical in meaning there should be only one representation of that meaning.
- It has been used by many programs that pretend to understand English.
- CD provides:

- a structure into which nodes representing information can be placed
 - a specific set of primitives
 - at a given level of granularity.
- Sentences are represented as a series of diagrams depicting actions using both abstract and real physical situations.
 - The agent and the objects are represented.
 - The actions are built up from a set of primitive acts which can be modified by tense.

Examples of Primitive Acts are:

ATRANS

-- Transfer of an abstract relationship. *e.g. give.*

PTRANS

-- Transfer of the physical location of an object. *e.g. go.*

PROPEL

-- Application of a physical force to an object. *e.g. push.*

MTRANS

-- Transfer of mental information. *e.g. tell.*

MBUILD

-- Construct new information from old. *e.g. decide.*

SPEAK

-- Utter a sound. *e.g. say.*

ATTEND

-- Focus a sense on a stimulus. *e.g. listen, watch.*

MOVE

-- Movement of a body part by owner. *e.g. punch, kick.*

GRASP

-- Actor grasping an object. *e.g. clutch.*

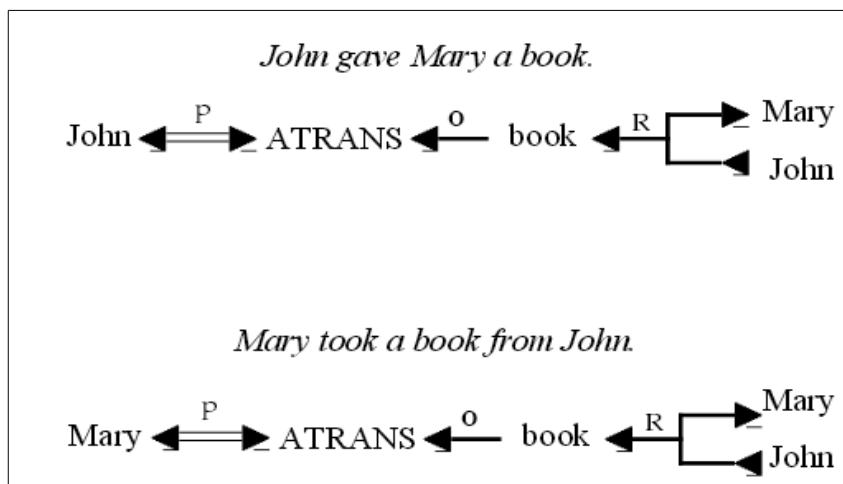
INGEST

-- Actor ingesting an object. *e.g. eat.*

EXPEL

-- Actor getting rid of an object from body. *e.g. throw*

Example:



SCRIPT

A script is a remembered precedent, consisting of tightly coupled, expectation-suggesting primitive-action and state change frames [Winston, 1992]

A script is a structured representation describing a stereotyped sequence of events in a particular context [Luger, Stubblefield, 1998, p.324]

That is, extend frames by explicitly representing expectations of actions and state changes.

Find primitives to describe the world like PTRANS for “transfer physical location of an object (= go)” and ATRANS for “transfer a relationship (= give)”.

A Restaurant Script

Script: RESTAURANT	
Track: Coffee Shop Props: Tables Menu F=Food Check Money Roles S=Customer W=Waiter C=Cook M=Cashier O=Owner	Entry cond.: S hungry S has money Results: S has less money O has more money S is not hungry S is pleased (optional)
Scene 1: Entering S PTRANS S into restaurant S ATTEND eyes to tables S MBUILD where to sit S PTRANS S to table S MOVE S to sitting position	Scene 2: Order (Menu on table) (W brings menu) S PTRANS menu to S S MTRANS food list to S S MTRANS signal to W W PTRANS W to table S MTRANS ‘I want F’ to W
Scene 3: Eating C ATRANS F to W W ATRANS F to S S INGEST F (Option: Return to Scene 2 to order more; otherwise, go to Scene 4)	Scene 4: Exiting W MOVE (write check) W PTRANS W to S W ATRANS check to S S ATRANS tip to S S PTRANS S to M S ATRANS money to M

	S PTRANS S to out of restaurant
--	---------------------------------

Chapter- 6 : Machine Learning

- Machine learning is a branch of AI that uses algorithm to allow computer to evolve behaviors based on data collected from databases or gathered through sensors.
- Machine learning focuses on prediction based on known properties learned from the training data.
- The performance is usually evaluated learned with respect to reproduce known knowledge.

Why Machine Learning?

- Recent Progress in algorithms and theory
- Huge computational power is available
- Many tasks would benefit from adaptive systems:
 - Robot exploring Mars (or cleaning your house!)
 - Software agents (OS functions, web searching)
 - Speech, vision, language, ...

Machine Learning Algorithms

- Supervised Learning
- Unsupervised Learning
- Semi-supervised Learning
- Reinforcement Learning (Environment provides feedback)
- Inductive learning
- Artificial Neural Network Learning
- Genetic Algorithm
- Bayesian Network..etc

Learning

- Learning is acquiring new or modifying existing knowledge, behaviors, skills and may involve synthesizing different types of information

Learning involves 3 factors:

Changes: Learning changes the learner. For machine learning the problem is determining the nature of these changes and how to best represent them.

Generalization: Learning leads to generalization. Performance must improve not only on the same task but on similar tasks

Improvement: Learning leads to improvements. Machine learning must address the possibility that changes may degrade performance and find ways to prevent it.

Learning Methods

- There are two different kinds of information processing which must be considered in a machine learning system
 - **Inductive learning** is concerned with determining general patterns, organizational schemes, rules, and laws from raw data, experience or examples.
 - **Deductive learning** is concerned with determination of specific facts using general rules or the determination of new general rules from old general rules.
 - $A=B, B=C, \text{ then } A=C$

Types of learning method

1. Rote Learning

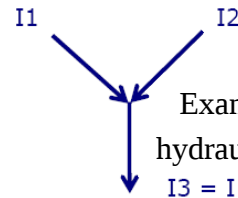
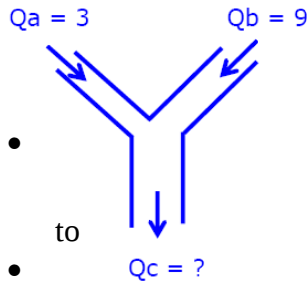
- It is a technique which focuses on memorization.
- Memorization - saving new knowledge to be retrieved when needed rather than calculated.
- It avoids understanding the inner complexities and inferences of the subject that is being learned.
- It works by taking problems that the performance element has solved and memorizing the problem and the solution.
- Only useful if it takes less time to retrieve the knowledge than it does to recompute it

Example: A.L. Samuels Checkers Player (1959-1967). It is a program that knows and follows the rules of checkers. It memorizes and recalls board positions it has encountered in previous games

2. Learning by Analogy

- Learning by analogy means acquiring new knowledge about an input entity by transferring it from a known similar entity.
- This technique transforms the solutions of problems in one domain to the solutions of the problems in another domain by discovering analogous states and operators in the two domains.

Examples of Analogy Learning



- to
- **Hydraulic Problem** Example: Infer by analogy the hydraulics laws that are similar Kirchhoff's laws.
 $I_3 = I_1 + I_2$ Kirchhoff's laws.
- Pressure drops like voltage drops
- **Kirchoff's First Law** Hydrogen atom is like our solar system. The Sun has a greater mass than the Earth and attracts it, causing the Earth to revolve around the Sun. The nucleus also has a greater mass than the electron and attracts it. Therefore it is plausible that the electron also revolves around the nucleus

3. Explanation Based Learning (EBL)

- Humans appear to learn quite a lot from one example.
- Human learning is accomplished by examining particular situations and relating them to the background knowledge in the form of known general principles.
- This kind of learning is called "Explanation Based Learning (EBL)".
- Tea cup example

4. Learning by Example (Inductive learning)

- **Learning by example** is a general learning strategy where a concept is learned by drawing inductive inferences from a set of fact.
- AI systems that learn by example can be viewed as searching a concept space by means of a decision tree.
- The best known approach to constructing a decision tree is called **ID3 (Iterative Dichotomizer 3)** developed by J. Ross Quinlan in 1975)

ID3

- ID3 is an algorithm used in decision tree learning to generate a decision tree.
- Decision tree consists of decision nodes and leaf nodes connected by arcs.
- ID3 builds the tree from the top down
- Entropy or Information gain is used to select the most useful attribute for classification
- **Entropy(H) = $-\sum (p_i) \log_2(p_i)$**
 - Entropy is the basis of Information Theory
 - Entropy is a measure of randomness, hence the smaller the entropy the greater the information content

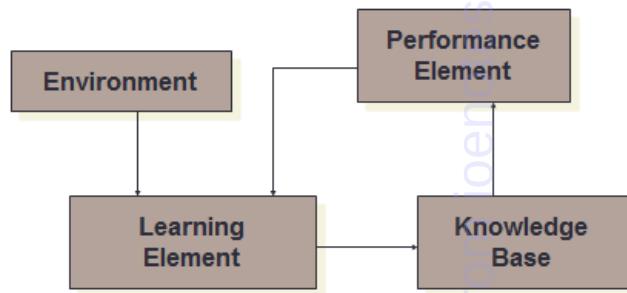
ID3 algorithm

- Create a root node for tree
- If all examples are positive, then create a positive node and stop
- If all examples are negative, then create a negative node and stop
- Otherwise
 - Calculate entropy, information gain to select root node and branch node.(nodes with highest information gain or minimum entropy is selected as root node)
 - Partition the examples into subset
 - Repeat until all examples are classified

Eg: Refer class note

Learning Framework

There are four major components in a learning system:



Environment

- The environment refers the nature and quality of information given to the learning element.
- The nature of information depends on its level (the degree of generality with respect to the performance element)
 - High level information is abstract, it deals with a broad class of problems
 - Low level information is detailed; it deals with a single problem.
- The quality of information involves
 - noise free
 - reliable
 - ordered

Learning Elements

- Acquire new knowledge through learning elements. Learning may be of

- Rote learning
- Learning by examples
- Learning by analogy
- Explanation based learning...etc
- The learning elements should have access to all internal actions of the performance element.

The Knowledge Base

The knowledge base should be

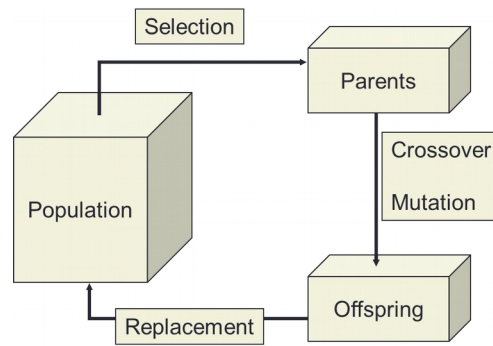
1. Expressive
 - Knowledge should be represented in understandable way
2. Modifiable
 - it must be easy to change the data in the knowledge base
3. Extendibility
 - The knowledge base must contain meta-knowledge (knowledge on how the data base is structured) so the system can change its structure

The Performance Element

- The performance element analyzes how complex the learning is and how learning is being performed?
- Complexity depends upon type of task. For learning, the simplest task is classification based on a single rule while the most complex task requires the application of multiple rules in sequence.
- The learning elements should have access to all internal actions of the performance elements.
- Transparency, the learning element should have access to all the internal actions of the performance element.

Genetic Algorithm

- A genetic algorithm maintains a population of candidate solutions for the problem at hand, and makes it evolve by iteratively applying a set of stochastic operators.
- It is a variation of stochastic beam search.
- Inspired by **biological evolution process**
- Uses concepts of “**Natural Selection**” i.e. “**Survival of the fittest**” and “**Genetic Inheritance**”
- Particularly well suited for hard problems where little is known about the underlying search space
- Widely used in business, science and engineering



Genetic process in Nature

Stochastic operators

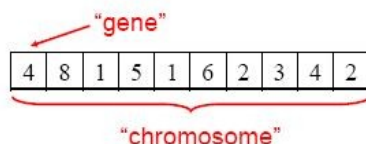
- **Selection** replicates the most successful solutions found in a population at a rate proportional to their relative quality
- **Crossover** decomposes two distinct solutions and then randomly mixes their parts to form novel solutions
- **Mutation** randomly produces a candidate solution.

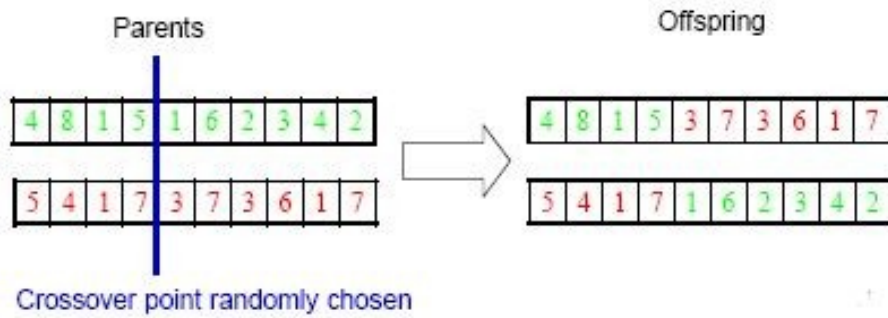
Comparison between Genetic Algorithm and Nature

Genetic Algorithm	Nature
Optimization problem	Environment
Feasible solutions	Individuals living in that environment
Solution Quality (fitness function)	Individual's degree of adaption to its surrounding environment
A set of feasible solutions	A population of organisms
Stochastic operators	Selection, Crossover and mutation in nature's evolutionary process
Iteratively applying a set of stochastic operators on a set of feasible solutions	Evolution of populations to suit their environment

Genetic Algorithm

- GA starts with k randomly generated states (population)
- A state is represented a string over a finite alphabet (often a string of 0s and 1s)
- Evaluation function (fitness function) defines fitness value of each states.
- Produce the next generation of states by selection, crossover, and mutation.
- The primary advantage of GA comes from crossover operation.

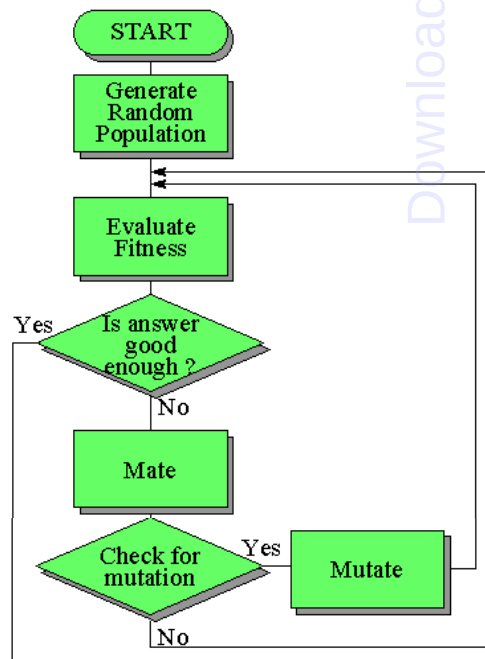




Algorithm

- produce an initial population of individuals
- evaluate the fitness of all individuals
- while (solution not found)
 - select fitter individuals for reproduction
 - recombine between individuals
 - mutate individuals
 - evaluate the fitness of the modified individuals
 - generate a new population
- End while

GA flowchart



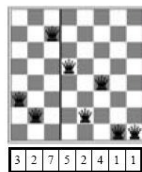
Disadvantage

- GA is better if the problem does not have any mathematical model for the solution.
- GA is less efficient in terms of speed of convergence.
- GA has tendency to get stuck in local maxima rather than global maxima.

An example

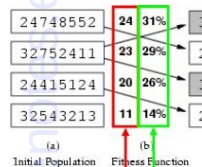
Example: 8-queens

- Fitness Function: number of nonattacking pairs of queens (28 is the value for the solution)
- Represent 8-queens state as an 8 digit string in which each digit represents position of queen

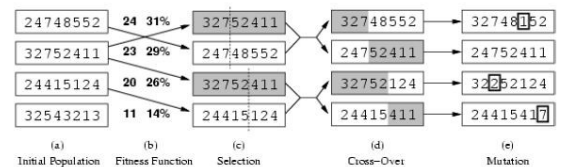


16

Example



Example: 8-queens (Selection)



Notice 3 2 7 5 2 4 1 1 is selected twice while
3 2 5 4 3 2 1 3 is not selected at all

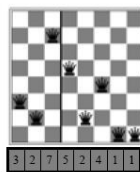
19

Fuzzy Learning

- In 1965 **Lotfi Zadeh**, published his famous paper “fuzzy sets”. Zadeh extended the work on possibility theory into a formal system of mathematical logic and introduced a new concept for applying natural language terms. This new logic for representing and manipulating fuzzy terms was called fuzzy logic.

- **Traditional Logic:** Traditional Boolean logic uses sharp distinctions. For instance Tom with height 181cm is tall. If we draw a line at 180 cm, David with height 179cm is small. Is David really small?

Example: 8



Example: 8-queens (Mutation)



Mutation corresponds to randomly selecting a queen and randomly moving it in its column

21

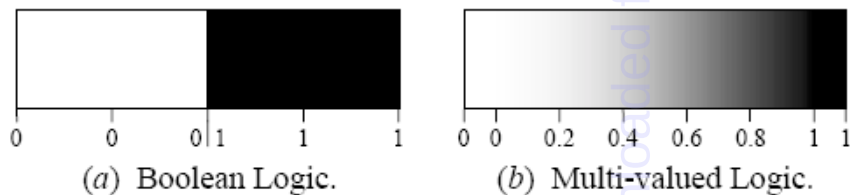
- **Fuzzy logic:** It is a form of knowledge representation suitable for notions that can't be defined precisely but which depend upon their contexts. A way to represent variation or imprecision in logic
- **Fuzzy** means “not clear, distinct or precise or blurred”. It is a concept of partial truth, where truth value may range between completely true or completely false.
- In contrast with traditional logic theory where binary sets have two valued logic (True/False), fuzzy logic variables may have value that ranges in degree from 0 to 1.
- Fuzzy logic is a form of multi-valued logic.
- Fuzzy logic reflects how people think. It attempts to model our sense of our decision making and our common sense.
- Example: Temperature, Height, Speed, Distance, Beauty
 - Motor is running really hot
 - Tom is a very tall guy

Crisp (Traditional) Variables

Crisp variables represent precise quantities. It denotes sharp distinctions.

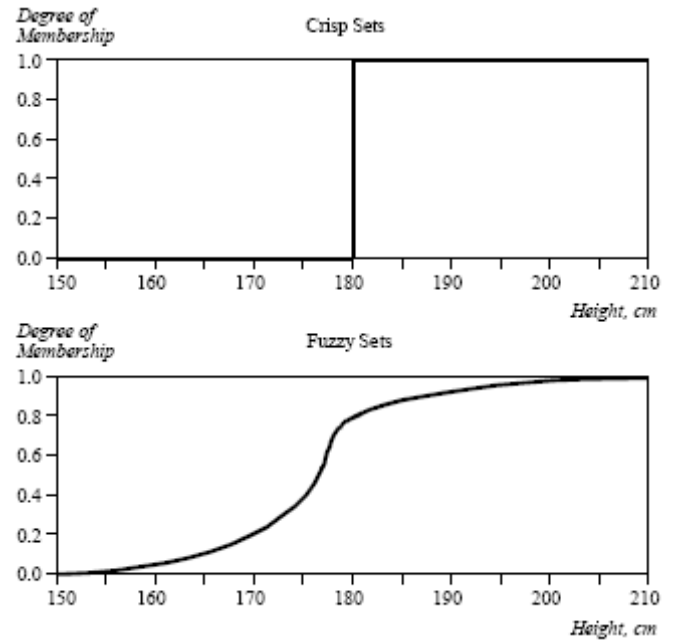
- $X = 3.1415$
- $A \in \{0,1\}$
- $Men \in \{Tall, short\}$
- $Speed \in \{slow, fast\}$

Range of logical values in Boolean and fuzzy logic



Crisp and fuzzy sets example

Name	Height, cm	Degree of Membership	
		Crisp	Fuzzy
Chris	208	1	1.00
Mark	205	1	1.00
John	198	1	0.98
Tom	181	1	0.82
David	179	0	0.78
Mike	172	0	0.24
Bob	167	0	0.15
Steven	158	0	0.06
Bill	155	0	0.01
Peter	152	0	0.00



- In fuzzy theory, fuzzy set A of universe X is defined by function $\mu_A(x)$ called membership function of set A.

$$\mu_A(x) : X \rightarrow [0,1],$$

Where, $\mu_A(x) = 1$, if x is totally in A

= 0, if x is not in A

$0 < \mu_A(x) < 1$, if x is partially in A.

- For any element x of X, membership function $\mu_A(x)$ equals the degree to which x is an element of set A. This degree ranges from 0 to 1, represents degree of membership, also called membership value of element x in set A.

$\mu_A : X \rightarrow [0,1]$ the membership function of A.

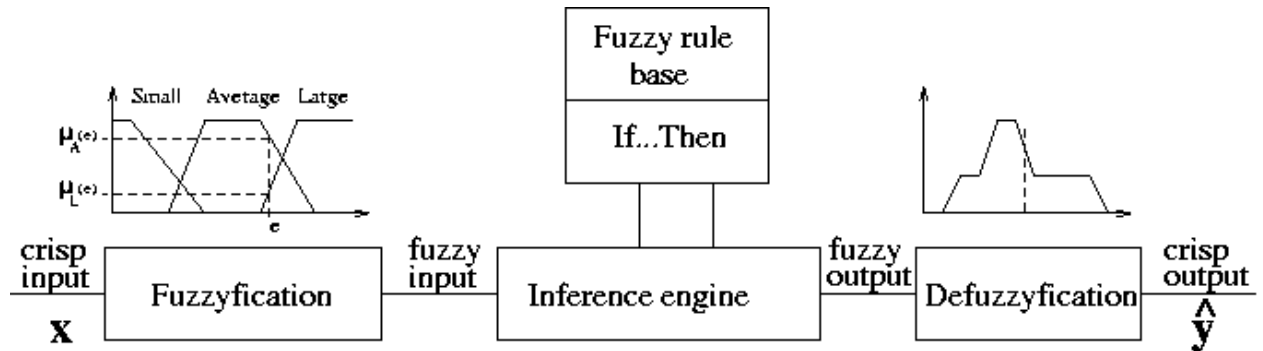
$\mu_A(x) \in [0,1]$ is the degree of membership x in A.

- A fuzzy variable is often denoted by its membership function.

Fuzzy Inferences

- Two approaches of fuzzy inference are
 - Mamdani Inference

- Sugeno fuzzy inference

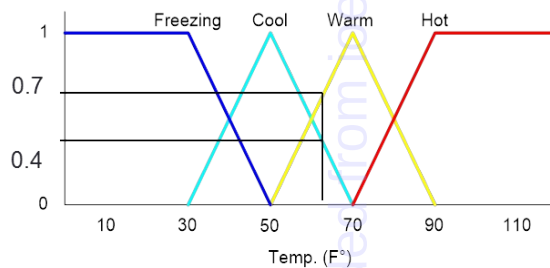


Mamdani inference applied in four stages:

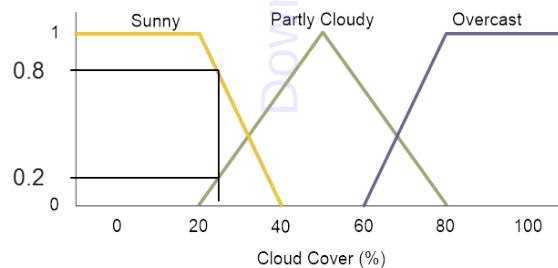
i. Fuzzyfication of input variables:

- Determines an input's membership in overlapping sets.
- Fuzzy Control combines the use of fuzzy linguistic variables with fuzzy logic

- $65\text{ F}^\circ \Rightarrow \text{Cool} = 0.4, \text{Warm} = 0.7$



- $25\% \text{ Cover} \Rightarrow \text{Sunny} = 0.8, \text{Cloudy} = 0.2$



ii. Rule Evaluation

- The second step is to take the fuzzified inputs, (such as $m_{(x=A1)} = 0.5$, $m_{(x=A2)} = 0.2$, $m_{(y=B1)} = 0.1$ and $m_{(y=B2)} = 0.7$), and apply them to the antecedents of the fuzzy rules.

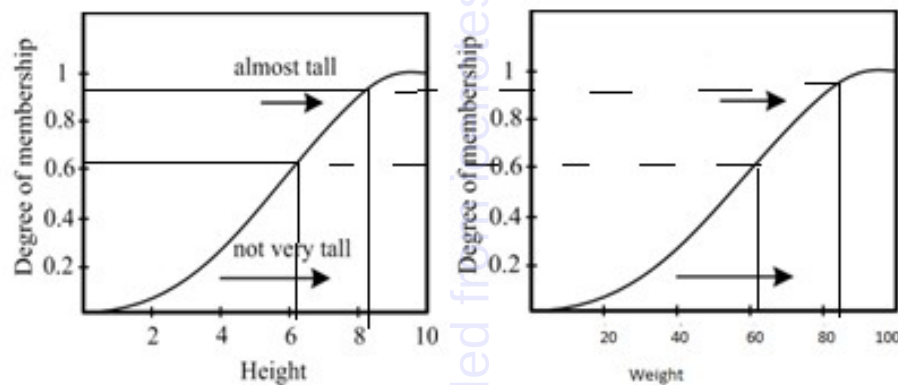
- If a given fuzzy rule has multiple antecedents, the fuzzy operator (AND or OR) is used to obtain a single number that represents the result of the antecedent evaluation.
- This number (the truth value) is then applied to the consequent membership function.

iii. Aggregation of the rule outputs:

- Aggregation is the process of unification of the outputs of all rules.
- Take the membership functions of all rule consequents previously scaled and combine them into a single fuzzy set.
- Determine outputs based on inputs and rules.
- The input of the aggregation process is the list of clipped or scaled consequent membership functions, and the output is one fuzzy set for each output variable.

iv. Defuzzification:

- Fuzziness helps us to evaluate the rules, but the final output of a fuzzy system has to be a crisp number.
- The input for the defuzzification process is the aggregate output fuzzy set and the output is a single number



Drawbacks

- Fuzzy logic deals with imprecision, and vagueness, but not uncertainty
- Requires tuning of membership functions
- Fuzzy Logic control may not scale well to large or complex problems

Boltzmann machine

- A **Boltzmann machine** is a type of stochastic recurrent neural network.
- A Boltzmann machine, like a Hopfield network, is a network of units with an "energy" defined for the network.

- Boltzmann machines can be seen as the stochastic, generative counterpart of Hopfield nets.
- First examples of a neural network capable of learning internal representations, and are able to solve difficult problems.
- Have not proven useful for practical problems in machine learning or inference.
- Theoretically exciting due to Hebbian nature of their training algorithm, as well as their parallelism and the resemblance of their dynamics to simple physical processes.
- If the connectivity is constrained, the learning can be made efficient enough to be useful for practical problems.
- The global energy, E , in a Boltzmann machine is identical in form to that of a Hopfield network:

$$E = -\left(\sum_{i < j} w_{ij} s_i s_j + \sum_i \theta_i s_i\right)$$

Where:

- w_{ij} is the connection strength between unit j and unit i .
- s_i is the state, $s_i \in \{0, 1\}$, of unit i .
- θ_i is the bias of unit i in the global energy function. ($-\theta_i$ is the activation threshold for the unit.)

The connections in a Boltzmann machine have two restrictions:

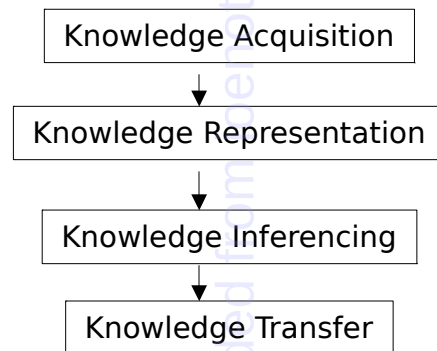
- $w_{ii} = 0 \quad \forall i$. (No unit has a connection with itself.)
- $w_{ij} = w_{ji} \quad \forall i, j$. (All connections are symmetric.)

Often the weights are represented in matrix form with a symmetric matrix W , with zeros along the diagonal.

Expert System

- An **expert system** is a computer system whose performance is guided by specific, expert knowledge in solving problems.
- It is a computer system that simulates the decision- making process of a human expert in a specific domain.
- Expert system is one of the early (large- scale) successes of artificial intelligence.
- An expert system is an “intelligent” program that solves problems in a narrow problem area by using high-quality, specific knowledge rather than an algorithm.
- Expert systems are used by most of the large or medium sized organization as a major tool for improving productivity and quality.
- An expert system’s knowledge is obtained from expert sources and code in a form suitable for the system to use in its process.

Steps in Expert System Development



Knowledge Acquisition:

- **Knowledge acquisition** is the process used to define the rules and ontologies required for a knowledge-based system.
- Describe the initial tasks associated with developing an expert system that include finding and interviewing domain experts and capturing their knowledge via rules, objects, and frame-based ontologies.
- The expert sources can be domain specialist, articles, journal, database etc.

Knowledge Representation:

- Representing information about the world in a form that a computer system can utilize to solve complex task.
- It is a set of ontological commitments i.e. an answer of question.
- Knowledge representation can be logical representation or structured representation.
- Representing the rules and constraints is through the use of logic rules, formally known as knowledge representation.

Knowledge Inferencing

- Knowledge inference refers to acquiring new knowledge from existing facts based on certain rules and constraints.
- Mostly rule based reasoning (Forward chaining/Backward chaining) is used for inferencing

Knowledge Transfer

- Knowledge transfer is the practical problem of transferring knowledge from one part of the organization to another.
- Knowledge transfer seeks to organize, create, capture or distribute knowledge and ensure its availability for future users.

Features of an Expert System

- Should have reasoning capacity. Reasoning may be goal-driven reasoning (backward chaining) or data-driven reasoning (forward chaining).
- Should cope with uncertainty.
- Should have proper knowledge representation i.e use knowledge rather than data.
- Should use symbolic representation for knowledge.
- Should use meta knowledge.
- Should use user interface.
- Should have ability to explain solutions with respect to problem specific.

Advantages of Expert System

- It provides consistent answer for repetitive decisions, processes and tasks.
- Hold and maintained significant level of information.
- Encourage organization to clarify the logic of their decision making.
- Ask question like human expertise.

Advantages of Expert System

- Lack of common sense needed in some decision making.
- Cannot make creative response as human expert would in unusual circumstances.
- Error may occur in the knowledge base and lead to wrong decision.
- Cannot adopt changing environment, unless knowledgebase is changed.

Applications

- Business
- Manufacturing
- Medicine
- Engineering
- Applied science
- Military
- Space
- Transportation
- Education
- Image analysis

- Chemical structure
- Agriculture
- Robotics and many more

History of Expert Systems

- Breakthrough in field of AI was achieved in area of Expert System
- 1966 - DENDRAL was developed at Stanford
 - Analyzes mass spectrographic, NMR, ... data and infers possible structures of unknown chemicals
- 1971 - MYCIN was developed at Stanford
 - diagnose and treat infectious blood diseases
- 1980 - XCON was developed at CMU
 - one of the first commercial expert systems
- PROSPECTOR: analyzing geological data
- GASOIL: designing gas-oil separation systems for offshore oil platforms

Advantages/Disadvantages (Comparison between human expert and Expert system)

Pro	Human expert	Expert system
Expert system	Perishable(decay)	Permanent
	Slow processing	Fast processing
	Unpredictable	Consistent
	Slow reproduction	Quick replication
	Expensive	Affordable
Pro	Broad focus	Narrow focus
	Inspiration	Lacks creative ability
	Adaptive needs	Instruction
	Common sense	Machine knowledge

Features of an ES

- Goal driven reasoning (backward chaining) or data driven reasoning (forward chaining)
- Coping with uncertainty
- Data representation
- User interface
- Explanations (ability to explain solution with respect to specific problem)
- Use knowledge rather than data
- Use symbolic representation for knowledge
- Should have meta knowledge

Components of ES

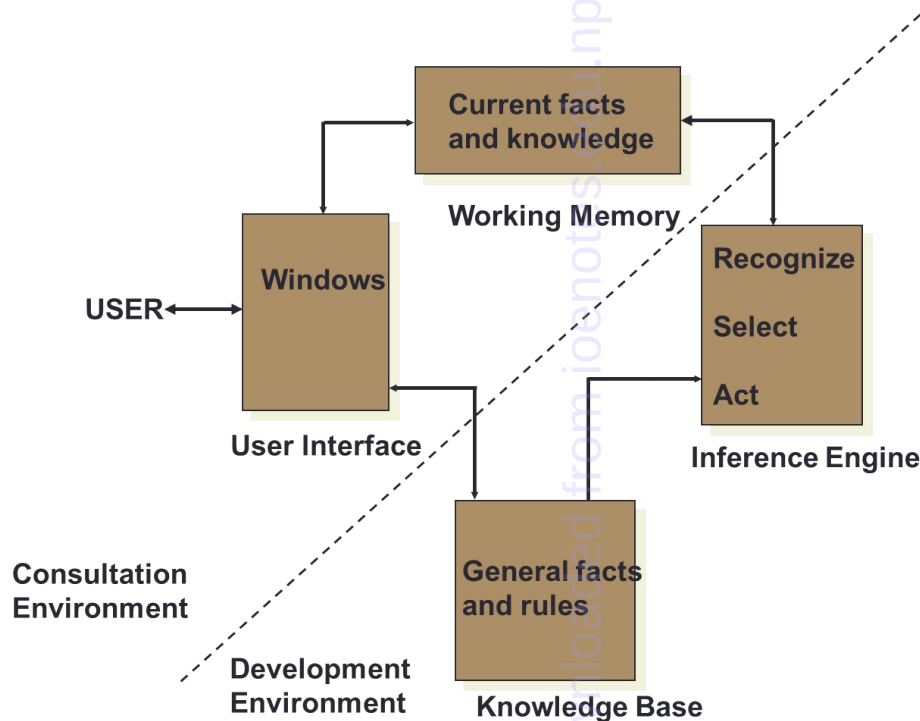
For a workable expert system more than a set of rules is required. We need four structures:

Knowledge Base - a data structure which contains the rules, often in IF THEN rules

Working Memory - a data structure which stores information about a specific problem

Inference Engine - a set of procedures for matching knowledge base with problem specific data in working memory

User Interface - Controls the dialog between user and the system



General Architecture of ES

Knowledge in expert systems is usually implemented as a **RULE**

A rule is an **IF-THEN** type statement

if <certain conditions are true>
then <take certain actions>

As a result, expert systems are sometimes called **rule-based systems or production systems**

Knowledge Base: Knowledge Engineering is the process of filling in an expert system knowledge base

Knowledge is information that has been:

- interpreted,
- categorised,
- applied, *experienced* and revised.

What it IS *NOT*:

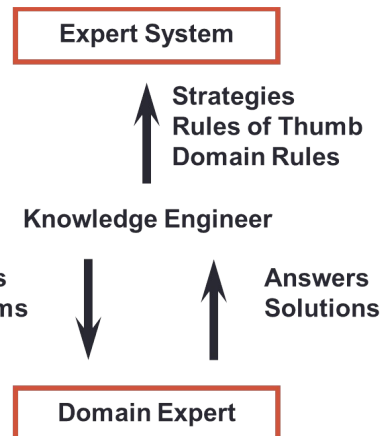
- Data, e.g. numbers.
- Information, i.e. interpreted data

Types of Knowledge

- **Procedural Knowledge:** "*Knowing How*", procedures
 - Includes step by step sequences and how-to types of instructions
 - It may contain explanations.
 - Eg. A mobile robot that navigates building contains procedures such as "navigate to room", "plan a path".
- **Declarative Knowledge:** "*Knowing what*",
 - Descriptive representation of knowledge
 - It tells us facts, about truths and associations. eg. "there is positive association between smoking and cancer"
 - Shallow, surface-level.
 - Eg. A mobile robot with facts to move forward, backward, turn left, turn right.
 - Important in initial stage of knowledge acquisition.
- **Episodic Knowledge:** *Experiential*
- **Meta-knowledge:** *Knowledge about Knowledge*

Sources of Knowledge

- Expert : primary source.
- End Users.
 - Usually have a good overview of the problem domain.
 - May provide valuable insight during initial investigations.
- Secondary / Tertiary Experts
 - Can provide specialised knowledge on sub-problems.
 - May give rise to conflicting advice.
- Literature
 - Reports, Guidelines, Books, Manuals etc.
 - Provide background and insight in early stages.



Knowledge Base Size

Frederick Hayes-Roth developed the following heuristics about knowledge bases:

- A convincing demonstration of a knowledge system's power requires about 250 rules.
- An expert level of competence in a narrow area requires about 500 to 1000 rules.
- Expertise in a profession requires about 10,000 rules.
- The limit of human expertise is about 100,000 rules.

Rule Types

- Relationship (*FACT*)
 - **IF** the battery is dead **THEN** the car will not start
- Recommendation
 - **IF** the car will not start **THEN** take a cab
- Directive
 - **IF** the car will not start **AND** the fuel system is ok **THEN** check out the electrical system
- Heuristic
 - **IF** the car will not start **AND** the car is a 1957 Ford **THEN** check the float

Rule Uncertainty

- Vagueness / uncertainty rules:
IF inflation is **HIGH**
THEN interest rates **might** be high
- Re-write using a confidence quantifier :
IF inflation is **HIGH**
THEN interest rates are high ($CQ = 0.8$)

Meta Rules

Rules that express knowledge about how other knowledge should be used.

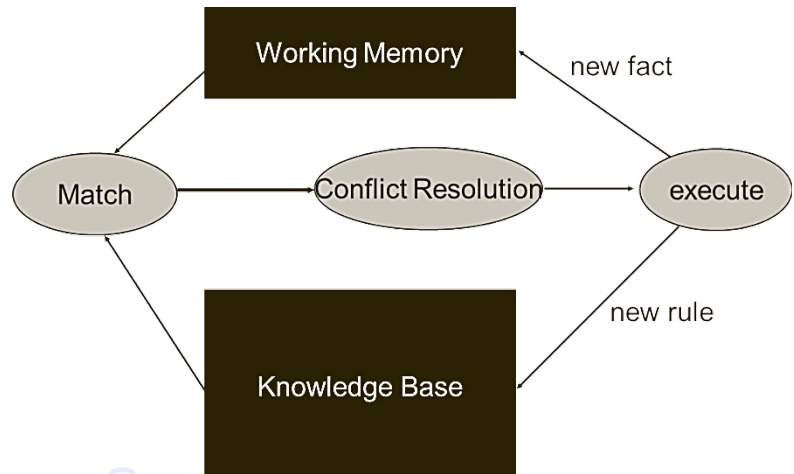
IF the car will not start
AND the electrical system is operating properly
THEN use fuel_system_rules

Inference Engine Structure

The inference engine is the mechanism for matching facts with rules and using the results to update the knowledge base. Most inference engines are based on the application of a logical reasoning rule called modus ponens which is stated as:

P1: if A then B

P2: A is true
Conclude: B is true



Recognize-Select-Act Cycle

It consists of 3 steps:

1. **Match:** Rules are compared to working memory to determine matches
2. **Conflict Resolution:** Select or enable a single rule for execution
3. **Execute:** Fire the selected rule

MYCIN

- Expert system for treating blood infections
- Diagnose patients based on reported symptoms and medical test results
- Could ask some more information and lab test results for diagnosis
- Recommend a course of treatment, if requested, MYCIN would explain the reasoning that lead to its diagnosis and recommendation.
- Use about 500 production rules, MYCIN operated roughly the same level of competence as human specialists in blood infections.
- Use backward chaining for reasoning.

DENDRAL

- First ES developed in late 1960
- Designed to analyze mass spectra
- Based on the mass of fragments seen in the spectra, it would be possible to make inference as the nature of molecule tested, identifying functional groups or even the entire molecule.
- Used Heuristic knowledge obtained from experienced chemists.
- Use forward chaining for reasoning

Neural Network

What are Neural Networks?

- Neural Networks are networks of neurons, for example, as found in real (i.e. biological) brains.
- Artificial neurons are crude approximations of the neurons found in real brains. They may be physical devices, or purely mathematical constructs.
- Artificial Neural Networks (ANNs) are networks of Artificial Neurons and hence constitute crude approximations to parts of real brains. They maybe physical devices, or simulated on conventional
- Computers point of view, an ANN is just a parallel computational system consisting of many simple processing elements connected together in a specific way in order to perform a particular task.

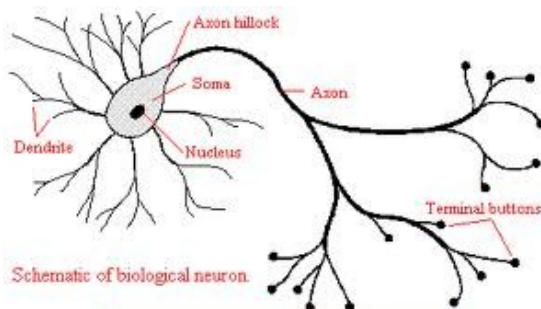
Why are Artificial Neural Networks?

- They are extremely powerful computational devices.
- Massive parallelism makes them very efficient
- They can learn and generalize from training data, so there is no need for enormous feats of programming
- They are particularly fault tolerant.
- They are very noise tolerant, so they can cope with situations where normal symbolic systems would have difficulty
- In principle, they can do anything a symbolic/logic system can do, and more

Brains vs. Computers

- **Processing elements:** There are 10 synapses in the brain, compared with 10 transistors in the computer
- **Processing speed:** 100 Hz for the brain compared to 10 Hz for the computer
- **Style of computation:** The brain computes in parallel and distributed mode, whereas the computer mostly serially and centralized.
- **Fault tolerant:** *The brain is fault tolerant, whereas the computer is not*
- **Adaptive:** The brain learns fast, whereas the computer doesn't even compare with an infant's learning capabilities
- **Intelligence and consciousness:** The brain is highly intelligent and conscious, whereas the computer shows lack of intelligence
- **Evolution:** The brains have been evolving for tens of millions of years; computers have been evolving for decades.

Basic Components of Biological Neurons



Neural Network

- The majority of neurons encode their activation or outputs as a series of brief electrical pulses (i.e. spikes or action potentials)
- The neuron's cell body (soma) processes the incoming activations and converts them into output activations.
- The neuron's nucleus contains the genetic material (DNA)
- Dendrites are fibers which emanate from the cell body and provide the receptive zone that receive activation from other neurons
- Axons are fibers acting as transmission lines that send action potentials to other neurons
- The junctions that allow signal transmission between the axons and the dendrites are called synapses.

The process of transmission is by diffusion of chemicals called neurotransmitters across the synaptic cleft.

Biological NN	Artificial NN
Soma	Neuron
Dendrite	Input
Axon	Output
Synapse	weight

What are Neural Networks used for?

There are two basic goals for neural network research:

Brain modeling:

- The biological goal of constructing models is of how real brains work.
- It can potentially help to understand the nature of perception, actions, learning and memory, thought and intelligence and/or formulate medical solutions to brain damaged patients.

Artificial System Construction:

- It may make machines more powerful and intelligent, relieve humans of tedious tasks, and may even improve upon human performance.

Weighting Factors (W)

The values $w_1, w_2, w_3, \dots, w_n$ are weights to determine the strength of input $x_1, x_2, x_3, \dots, x_n$

- The neuron computes the weighted sum of the input signals and compares the result with a threshold value, ω .

Neural Network

$$\bullet \quad X = x_1w_1 + x_2w_2 + x_3w_3 + \dots + x_nw_n = \sum_{i=1}^n x_iw_i$$

- If the net input is less than the threshold (ω), the neuron output is -1.
- If the net input is greater than or equal to the threshold(ω) , the neuron becomes activated and its output attains value +1.

Threshold (ω)

Threshold is the magnitude offset value of the node. It affects the activation of the node output Y as

$$Y = f(X) = f \left\{ \sum_{i=1}^n x_iw_i - \omega \right\}$$

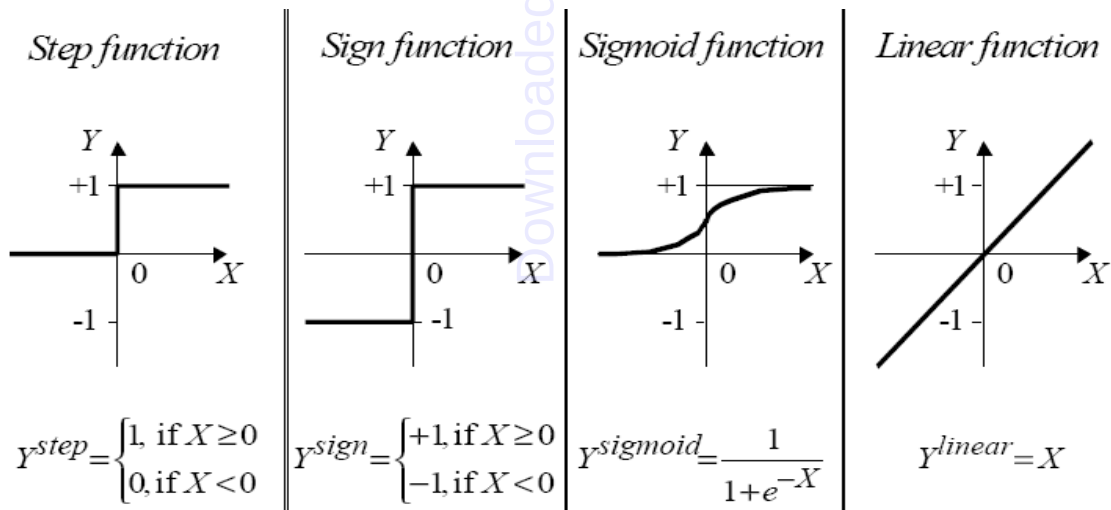
In practice neurons generally do not fire unless their total input goes above a threshold value.

Activation function

An activation function f performs a mathematical operation on the signal output. Activation function in example below is called sign function.

$$X = \sum_{i=1}^n x_iw_i \quad Y = \begin{cases} +1, & \text{if } X \geq \theta \\ -1, & \text{if } X < \theta \end{cases}$$

Different types of Activation Function are given as figure below



Neural Network

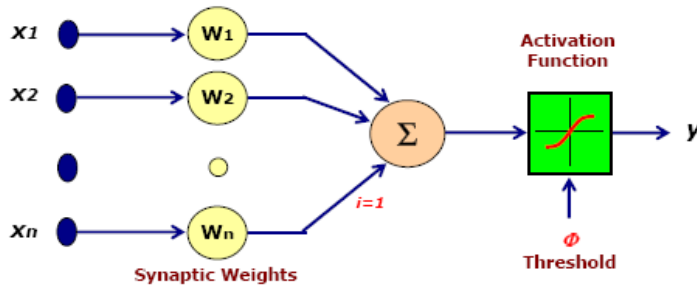


Fig Basic Elements of an Artificial Linear Neuron

Types of Neural Network

1. Single layer feed forward network

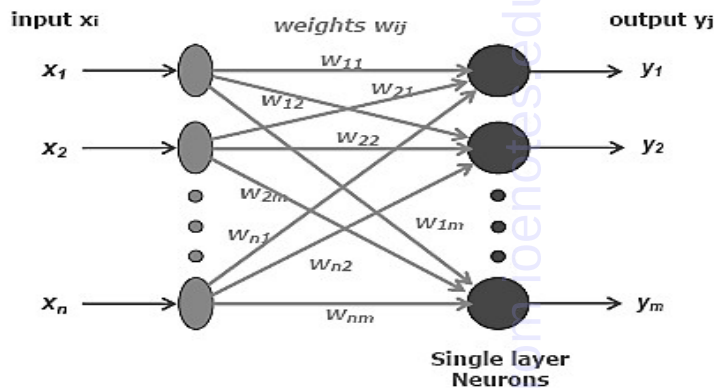


Fig. Single Layer Feed-forward Network

2. Multilayer feed forward network

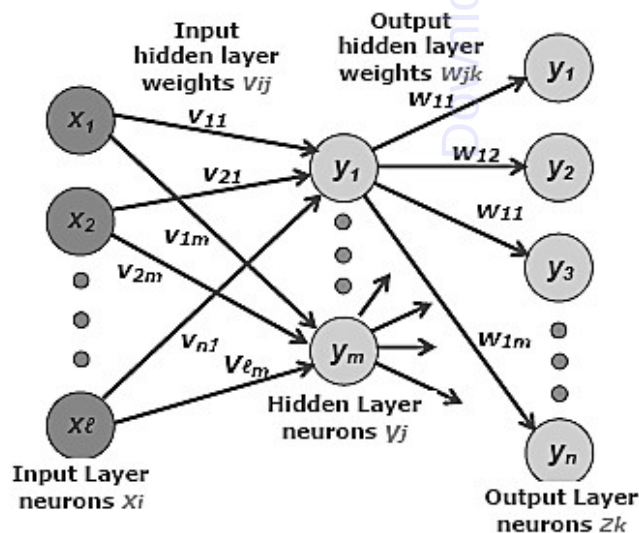


Fig. Multilayer feed-forward network in $(\ell - m - n)$ configuration.

Neural Network

3. Recurrent network (feedback)

Example :

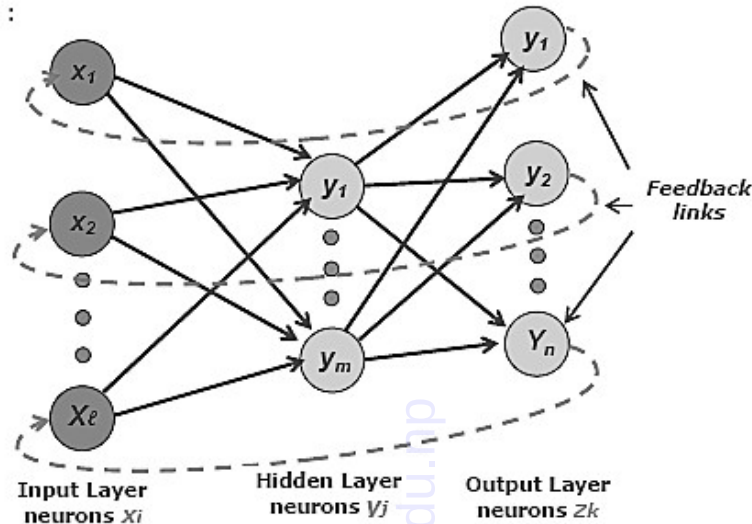


Fig. Recurrent neural network

Advantage of NN

- NN can perform tasks that a linear program cannot perform
- NN help where we can not formulate an algorithmic solution or where we get lots of examples of the behavior we require.
- When an element of the NN fails, it can continue without any problem by their parallel nature because of their distributive information nature.
- NN learns and does not need to be reprogrammed.
- Has massive parallel processing.
- Widely applied in data classification, clustering, pattern recognition, etc

Neural Network Applications

- **Brain modeling:** Aid our understanding of how the brain works, how behavior emerges from the interaction of networks of neurons, what needs to "get fixed" in brain damaged patients
- **Real world applications**
 - Financial modeling – predicting the stock market
 - Time series prediction – climate, weather, seizures
 - Computer games – intelligent agents, chess, backgammon
 - Robotics – autonomous adaptable robots
 - Pattern recognition – speech recognition, seismic activity, sonar signals
 - Data analysis – data compression, data mining

Neural Network

- Bioinformatics – DNA sequencing, alignment

Learning Processes in Neural Networks

- Neural network has the ability to learn from its environment, and to improve its performance through learning.
- The improvement in performance takes place over time in accordance with some prescribed measure.
- A neural network learns about its environment through an iterative process of adjustments applied to its synaptic weights and thresholds.
- The network becomes more knowledgeable about its environment after each iteration of the learning process.

There are three broad types of learning:

1. Supervised learning (i.e. learning with an external teacher)
2. Unsupervised learning (i.e. learning with no help)
3. Reinforcement learning (i.e. learning with limited feedback)

Supervised learning

- Supervised learning is the [machine learning](#) task of inferring a function from training data and the [training data](#) consist of a set of training examples i.e. a supervised learning algorithm analyzes the training data and produces an inferred function, which can be used for mapping new examples.
- In supervised learning, each example is a pair consisting of an input object and a desired output value. In other words, the inputs are assumed to be at the beginning and outputs at the end of the causal chain.
- In supervised the variables under investigation can be split into two groups: explanatory variables and one (or more) dependent variables.
- The target of the analysis is to specify a relationship between the explanatory variables and the dependent variable as it is done in regression analysis.
- Approaches to unsupervised learning include: Classification

Unsupervised Learning

- Unsupervised learning is the task of finding hidden structure in unlabeled data. Since the examples given to the learner are unlabeled, there is no error or reward signal to evaluate a potential solution.
- In unsupervised learning situations all variables are treated in the same way, there is no distinction between explanatory and dependent variables.
- For unsupervised learning typically either the target variable is unknown or has only been recorded for too small a number of cases.
- Unsupervised learning is closely related to the problem of [density estimation](#) in [statistics](#).
- Approaches to unsupervised learning include: [Clustering](#)

Learning Rate (α)

Neural Network

- Learning rate is a constant in the algorithm of a neural network that affects the speed of learning.
- It is a control parameter of some training algorithms, which controls the step size when weights are iteratively adjusted.
- The performance of the algorithm is very sensitive to the proper setting of the learning rate.
- If the learning rate is set too high, the algorithm may oscillate and become unstable whereas if the learning rate is too small, the algorithm will take too long to converge.
- It is not practically significant to determine the optimal setting for the learning rate before training. In most of the cases the optimal learning rate changes during the training process.

Backpropagation Algorithm

1. **Initialization:** Set all the weights and threshold levels of the n/w to random numbers uniformly distributed inside a small range $[-2.4/T_i, 2.4/T_i]$ where T_i is the total number of inputs of neuron i in the network
2. **Activation:** Activate the back-propagation neural network by applying inputs $x_i(p)$ and desired outputs $y_i(p)$

- Calculate the actual outputs of the neuron in the hidden layer.

$$Y_j(p) = \text{sigmoid} \left[\sum_{i=0}^n x_i(p) \cdot w_{ij}(p) - \theta_j \right]$$

- Calculate the actual output of the neurons in the output layer

$$Y_k(p) = \text{sigmoid} \left[\sum_{j=0}^m x_{jk}(p) \cdot w_{jk}(p) - \theta_k \right]$$

- Where m is the number of inputs of neuron k in the output layer

3. **Weight training:** Update weights in the back-propagation network by propagating backward the errors associated with output neurons

- Calculate the error gradient of the neurons in the output layer

$$\delta_k = y_k(p) [1 - y_k(p)] \cdot e_k(p) \text{ where } e_k(p) = y_{d,k}(p) - y_k(p)$$

- Calculate the weight corrections

$$\Delta w_{jk} = \alpha \cdot y_j(p) \cdot \delta_k(p)$$

- Update the weight at the output neurons

$$w_{jk}(p+1) = w_{jk}(p) + \Delta w_{jk}(p)$$

- Calculate the error gradient for the neurons in the hidden layer

$$\delta_j(p) = y_j(p) [1 - y_j(p)] \cdot \left[\sum_{k=1}^l \delta_k(p) w_{jk}(p) \right]$$

- Calculate the weight corrections $\Delta w_{ij} = \alpha \cdot x_i(p) \cdot \delta_j(p)$

- Update the weight at the hidden neuron

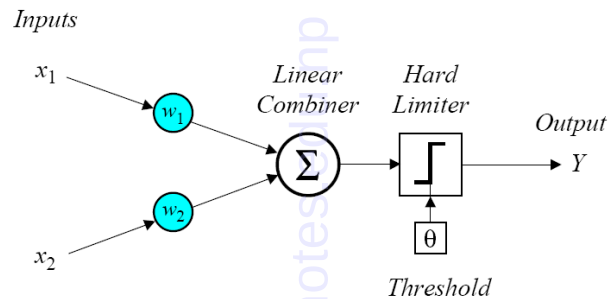
$$w_{ij}(p+1) = w_{ij}(p) + \Delta w_{ij}(p)$$

Neural Network

4. **Iteration:** Increase iteration by one, go back to step 2 and repeat the process until the selected error criterion is satisfied.

PERCEPTRON

- A Perceptron is the simplest kind of feed forward neural network invented by Frank Rosenblatt
- A perceptron can learn any linearly separable functions, given enough training.
- The model consists of a linear combiner followed by a activation function.
- The weighted sum of the inputs is applied to the activation function, which produces an output equal to +1 if its input is positive and -1 if it is negative.



Single layer 2 input Perceptron model

Perceptron algorithm

1. Initialization

- a. Set the initial weights w_i and threshold θ to random numbers in the range $[-0.5, +0.5]$
- b. If the error, $e(p)$ is positive, we need to increase perceptron output $Y(p)$, but if it is negative, we need to decrease $Y(p)$

2. Activation

- a. Activate the perceptron by applying inputs $x_i(p)$ and desired output $Y_d(p)$. Calculate the actual output at iteration $p=1$

Where n is the number of the perceptron inputs, and step is activation function

$$Y(p) = \text{step} \left[\sum_{i=1}^n x_i(p) w_i(p) - \theta \right] \text{step}$$

3. Weight Training

- a. Update the weights of the perceptron
 $W_i(p+1) = w_i(p) + \Delta w_i(p)$, Where $\Delta w_i(p)$ is the weight correction at iteration p . the weight correction is computed by the delta rule.
 $\Delta w_i(p) = \alpha \cdot X_i(p) \cdot e(p)$, α is learning rate

4. Iteration

- a. Increase iteration p by one, go back to Step 2 and repeat the process until convergence.

Neural Network

Perceptron example: *construct Perceptron neural network model that performs like AND gate, with learning rate $\alpha = 0.1$ and threshold $\omega = 0.2$*

HEBBIAN LEARNING

The oldest and most famous of all learning rules is Hebb's postulate of learning:

“When an axon of cell A is near enough to excite a cell B and repeatedly or persistently takes part in firing it, some growth process or metabolic changes take place in one or both cells such that A's efficiency as one of the cells firing B is increased”

It is applied to artificial neurons as “If two interconnected neurons are both “on” at the same time, then the weight between them should be increased”

The operation of a NN is determined by the values of the interconnection weights

There is no algorithm that determines how the weight should be assigned in order to solve a specific problem. Hence weights are determined by a learning process

Hebb's Algorithm

- **Step 0:** initialize all weights to 0
- **Step 1:** Given a training input, s , with its target output, t , set the activations of the input units: $x_i = s_i$
- **Step 2:** Set the activation of the output unit to the target value: $y = t$
- **Step 3:** Adjust the weights: $w_i(\text{new}) = w_i(\text{old}) + x_i y$
- **Step 4:** Adjust the bias (just like the weights): $b(\text{new}) = b(\text{old}) + y$

Example: *Construct a Hebb Net which performs like an AND function, that is, only when both features are “active” will the data be in the target class.*

Downloaded from ioenotes.edu.np

Neural Network

AND gate.



x_1	x_2	bias	target
1	1	1	1
1	-1	1	-1
-1	1	1	-1
-1	-1	1	-1

→ Initialise weights and bias = 0

→ present first i/p (1, 1, 1) with target o/p = 1

update weight as $w_1(\text{new}) = w_1(\text{old}) + x_1 t$
 $= 0 + 1 \cdot 1 = 1$

$w_2(\text{new}) = 0 + 1 \cdot 1 = 1$

$b(\text{new}) = 0 + 1 = 1$

→ present 2nd i/p (1, -1, 1) with target o/p = -1.

update weight as, $w_1(\text{new}) = 1 + 1(-1) = 0$

$w_2(\text{new}) = 1 + (-1)(-1) = 2$

$b(\text{new}) = 1 + (-1) = 0$

→ present 3rd i/p (-1, 1, 1) with target o/p = -1.

update weight as: $w_1(\text{new}) = 0 + (-1)(-1) = 1$

$w_2(\text{new}) = 2 + (1)(-1) = 1$

$b(\text{new}) = 0 + (-1) = -1$

→ present 4th i/p (-1, -1, 1) with target o/p = 1

update weight as: $w_1(\text{new}) = 1 + (-1)(-1) = 2$

$w_2(\text{new}) = 1 + (-1)(-1) = 2$

$b(\text{new}) = (-1) + (-1) = -2$

Now this neuron works with $w_1 = 2$, $w_2 = 2$, $b = -2$

Testing with all possible input data.

$$1 \times 2 + 1 \times 2 + 1 \times (-2) = 2 > 0 \Rightarrow 1$$

$$1 \times 2 + 1 \times 2 + 1 \times (-2) = -2 < 0 \Rightarrow -1$$

$$1 \times 2 + (-1) \times 2 + 1 \times (-2) = -2 < 0 \Rightarrow -1$$

$$-1 \times 2 + (-1) \times 2 + 1 \times (-2) = -6 < 0 \Rightarrow -1$$

Neural Network

ADALINE NETWORK

Adaline network is a variation on the Perceptron Network

- inputs are +1 or -1
- outputs are +1 or -1
- uses a bias input

It is trained using the Delta Rule which is also known as the least mean squares (LMS) or Widrow-Hoff rule. The activation function, during training is the **identity function**. After training the activation is a threshold function

Adaline Algorithm

- **Step 0:** initialize the weights to small random values and select a learning rate, α
- **Step 1:** for each input vector s , with target output, t set the inputs to s
- **Step 2:** compute the neuron inputs

Neuron input

$$y_{in} = b + \sum x_i w_i$$

Delta rule

$$\begin{aligned} b(\text{new}) &= b(\text{old}) + \alpha(t - y_{in}) \\ w_i(\text{new}) &= w_i(\text{old}) + \alpha(t - y_{in})x_i \end{aligned}$$

- **Step 3:** use the delta rule to update the bias and weights
- **Step 4:** stop if the largest weight change across all the training samples is less than a specified tolerance, otherwise cycle through the training set again

Learning Rate for adaline network

- The performance of an ADALINE neuron depends heavily on the choice of the learning rate
 - if it is too large the system will not converge
 - if it is too small the convergence will take to long
- Typically, α is selected by trial and error
 - typical range: $0.01 < \alpha < 10.0$
 - often start at 0.1
 - sometimes it is suggested that:
 $0.1 < n \alpha < 1.0$

where n is the number of inputs

Example: Construct an AND function for a ADALINE neuron. let $\alpha = 0.1$

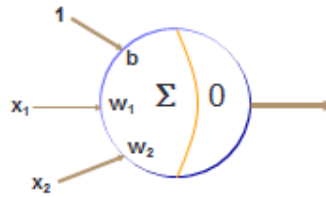
$$y_{in} = b + \sum x_i w_i$$

Activation Function

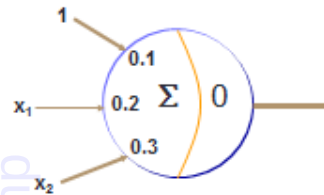
$$y = \begin{cases} +1 & \text{if } y_{in} \geq 0 \\ -1 & \text{if } y_{in} < 0 \end{cases}$$

Neural Network

x1	x2	bias	Target
1	1	1	1
1	-1	1	-1
-1	1	1	-1
-1	-1	1	-1

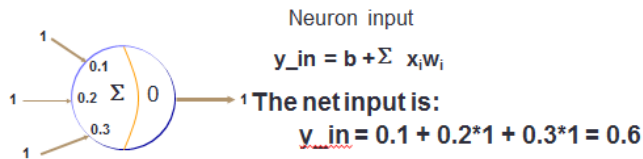


Initial Conditions: Set the weights to small random values:



First Training Run

- Apply the input (1,1) with output 1



Delta rule

$$b(\text{new}) = b(\text{old}) + \alpha(t - y_{in})$$

$$w_i(\text{new}) = w_i(\text{old}) + \alpha(t - y_{in})x_i$$

The new weights are:

$$b = 0.1 + 0.1(1 - 0.6) = 0.14$$

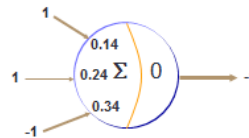
$$w_1 = 0.2 + 0.1(1 - 0.6)1 = 0.24$$

$$w_2 = 0.3 + 0.1(1 - 0.6)1 = 0.34$$

The largest weight change is 0.04

Second Training Run

- Apply the second training set (1 -1) with output -1



The net input is:

$$y_{in} = 0.14 + 0.24*1 + 0.34*(-1) = 0.04$$

The new weights are:

$$b = 0.14 - 0.1(1 + 0.04) = 0.04$$

$$w_1 = 0.24 - 0.1(1 + 0.04)1 = 0.14$$

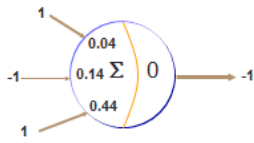
$$w_2 = 0.34 + 0.1(1 + 0.04)1 = 0.44$$

The largest weight change is 0.1

Neural Network

Third Training Run

- Apply the third training set (-1 1) with output -1



The net input is:

$$y_{in} = 0.04 - 0.14*1 + 0.44*1 = 0.34$$

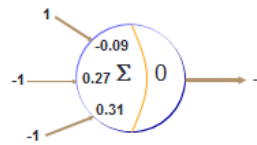
The new weights are:

$$\begin{aligned} b &= 0.04 - 0.1(1+0.34) = -0.09 \\ w_1 &= 0.14 + 0.1(1+0.34)1 = 0.27 \\ w_2 &= 0.44 - 0.1(1+0.34)1 = 0.31 \end{aligned}$$

The largest weight change is 0.13

Fourth Training Run

- Apply the fourth training set (-1 -1) with output -1



The net input is:

$$y_{in} = -0.09 - 0.27*1 - 0.31*1 = -0.67$$

The new weights are:

$$\begin{aligned} b &= -0.09 - 0.1(1+0.67) = -0.27 \\ w_1 &= 0.27 + 0.1(1+0.67)1 = 0.43 \\ w_2 &= 0.31 + 0.1(1+0.67)1 = 0.47 \end{aligned}$$

The largest weight change is 0.16

Continue to cycle through the four training inputs until the largest change in the weights over a complete cycle is less than some small number (say 0.01)

In this case, the solution becomes

- $b = -0.5$
- $w_1 = 0.5$
- $w_2 = 0.5$

Kohonen Nwtwork

- The Kohonen neural network differs from the feedforward back propagation neural network.
- Kohonen neural networks are used because they are a relatively simple network to construct that can be trained very rapidly.
- The Kohonen neural network contains only an input and output layer of neurons. There is no hidden layer in a Kohonen neural network
- It differs both in how it is trained and how it recalls a pattern. The Kohonen neural network does not use any sort of activation function and a bias weight.
- Output from the Kohonen neural network does not consist of the output of several neurons.
- When a pattern is presented to a Kohonen network one of the output neurons is selected as a "winner". This "winning" neuron is the output from the Kohonen network.
- Often these "winning" neurons represent groups in the data that is presented to the Kohonen network.

Neural Network

- The most significant difference between the Kohonen neural network and the feed forward back propagation neural network is that the Kohonen network trained in an unsupervised mode.

Difference between the Kohonen neural network and the feed forward back propagation neural network:

- Kohonen network trained in an unsupervised mode.
- This means that the Kohonen network is presented with data, but the correct output that corresponds to that data is not specified.
- Using the Kohonen network this data can be classified into groups.

Limitations of the Kohonen neural network:

- The neural networks with only two layers can only be applied to linearly separable problems and this is the case with the Kohonen neural network.
- Kohonen neural networks are used because they are a relatively simple network to construct that can be trained very rapidly.

Hopfield Neural Network

- A Hopfield network is a form of [recurrent artificial neural network](#) invented by [John Hopfield](#).
- They are guaranteed to converge to a [local minimum](#), but convergence to a false pattern (wrong local minimum) rather than the stored pattern (expected local minimum) can occur. Hopfield networks also provide a model for understanding human memory.
- The Hopfield neural network is perhaps the simplest of neural networks. The Hopfield neural network is a fully connected single layer auto associative network. This means it has one single layer, with each neuron connected to every other neuron.
- The units in Hopfield nets are binary threshold units, i.e. the units only take on two different values for their states and the value is determined by whether or not the units' input exceeds their threshold. Hopfield nets normally have units that take on values of 1 or -1.
- Hopfield network consists of a set of N interconnected neurons which update their activation values asynchronously and independent of other neurons.

Single layer n neuron Hopfield network

Primary application of the hopfield network is an associative memory. Weights of the connections between the neurons have to be set such that it corresponds to some pattern.

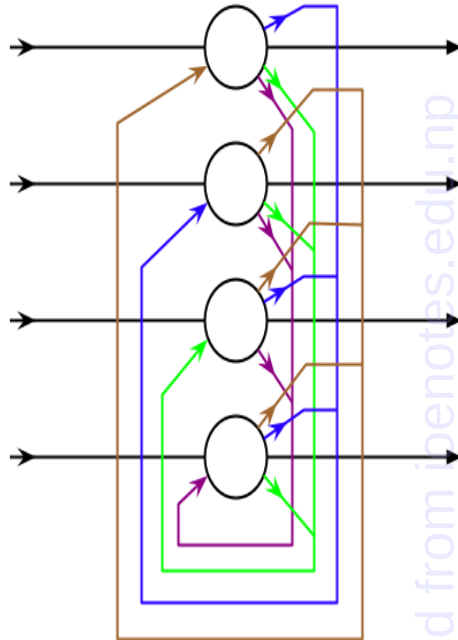
- Hopfield chose activation values of 1 and 0, but now we use values +1 and -1
- The net input $s_k(t+1)$ of a neuron k at cycle t+1 is a weighted sum

$$s_k(t+1) = \sum_{j \neq k} y_j(t) w_{jk} + \theta_k.$$

Neural Network

A simple threshold function is applied to the net input to obtain the new activation value $y_k(t + 1)$ at time $t + 1$:

$$y_k(t + 1) = \begin{cases} +1 & \text{if } s_k(t + 1) > U_k \\ -1 & \text{if } s_k(t + 1) < U_k \\ y_k(t) & \text{otherwise,} \end{cases}$$

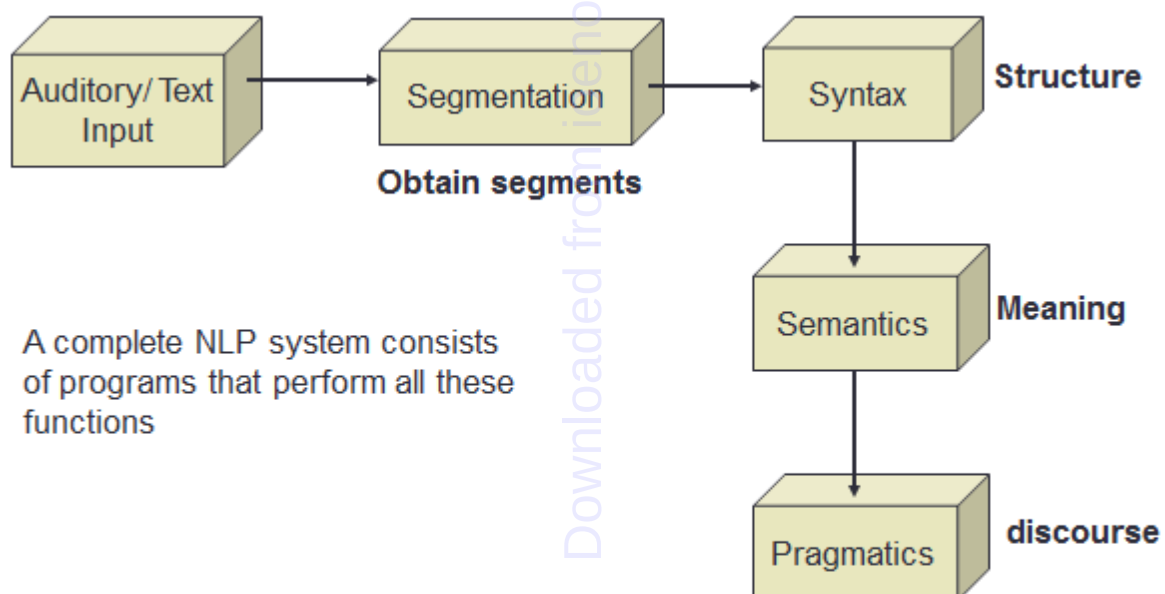


7

Natural Language Processing

- Language is means of Communication for humans. By studying language, we come to understand more about the world.
- If we can succeed at building computational mode of language, we will have a powerful tool for communicating about the world.
- We look at how we can exploit knowledge about the world, in combination with linguistic facts, to build computational natural language systems.
- Natural Language Processing (NLP) is the process of computer analysis of input provided in a human language (natural language), and conversion of this input into a useful form of representation.
- NLP is one of field of AI that processes or analyzes written or spoken language.
- NLP involve processing of speech, grammar and meaning.
- NLP is composed of two part: NLU (Natural Language Understanding) and NLG (Natural Language generation).
- Understanding any language requires detailed knowledge of that language.

NLP Steps/ Processes



Input/Source

- The input of a NLP system can be written text or speech.
- Quality of input decides the possible errors in language processing that is high quality input leads to correct language understanding.

Segmentation

- The text inputs are divided in segments (Chunks) and the segments are analyzed. Each such chunk is called frames.

Syntactic Analysis

- Syntactic analysis takes an input sentence and produces a representation of its grammatical structure.
- A grammar describes the valid parts of speech of a language and how to combine them into phrases.
- The grammar of English is nearly context free.

Grammar: A computer grammar specifies which sentences are in a language and their parse trees. A parse tree is a hierarchical structure that shows how the grammar applies to the input. Each level of the tree corresponds to the application of one grammar rule.

Syntax

1. sentence $\leftrightarrow$ noun_phrase verb_phrase
2. noun_phrase $\leftrightarrow$ noun
3. noun_phrase $\leftrightarrow$ article noun
4. verb_phrase $\leftrightarrow$ verb
5. verb_phrase $\leftrightarrow$ verb noun_phrase
6. article $\leftrightarrow$ a
7. article $\leftrightarrow$ the
8. noun $\leftrightarrow$ man
9. noun $\leftrightarrow$ dog
10. verb $\leftrightarrow$ likes
11. verb $\leftrightarrow$ bites

Parse tree

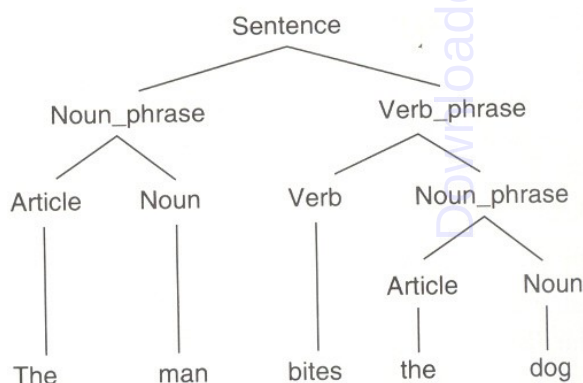


Figure 10.3 Parse tree for the sentence "The man bites the dog."

Semantic Analysis

- Semantic analysis is a process of converting the syntactic representations into a meaning representation.
- This involves the following tasks:

- Word sense determination
- Sentence level analysis

Word sense: Words have different meanings in different contexts.

Example: *Mary had a bat in her office.*

bat = 'a baseball thing'

bat = 'a flying mammal'

Sentence Level Meaning

Once the words are understood, the sentence must be assigned some meaning

Examples:

- She saw her duck.
- I saw a man with a telescope.

Non-examples: Colorless green ideas sleep furiously - > This would be rejected semantically as *colorless green* would make no sense

Pragmatic Analysis

- Pragmatics comprises aspects of meaning that depend upon the context or upon facts about real world.
- These aspects include:
 - Pronouns and referring expressions
 - Logical inferences, that can be drawn from the meanings of a set of propositions.
 - Discourse structure: the meaning of a collection of sentences taken together.

Examples

- **Jack fell. Jill brought him a band-aid.**
Jack got hurt and Jill wanted to help.
- **We got seven letters today**
We got only seven letters (and not eight).
- **Understanding toothpaste directions.**
For best results, squeeze tube from the bottom and flatten as you go up.
 - The tube has an opening at only one end.
 - The tube can be squeezed,
 - The instructee wants to use the paste.
 - The tube can be emptied only through its mouth.
- Lexicon or dictionary
- Morphological Analysis system

NLP AIDS

- To understand or analyze language some aids or tools can be used in NLP

i. Lexicon or Dictionary

- A lexicon defines the words of a language that a system knows about.
- This includes common words and words that are specific to the domain of the application.
- Entries include meanings for each word and its syntactic and morphological behavior.

ii. **Morphological Analysis System**

- Morphological analysis is the process of recognizing the suffixes and prefixes that have been attached to a word.
- We do this by having a table of affixes and trying to match the input as: prefixes+root+suffixes.
 - o For example: adjective + ly -> adverb
 - o We may not get a unique result.
 - o “-s, -es” can be either a plural noun or a verb
 - o “-d, -ed” can be either a past tense or a perfect participle
- **Morphological Information**
- Transform part of speech
 - o *green, greenness (adjective, noun)*
 - o *walk, walker (verb, noun)*
- Change features of nouns
 - o *boat, boats (singular, plural)*
- Bill slept , Bill’s bed
 - o (subjective case, possessive case)
- Change features of verbs
 - o Aspect
 - *I walk. I am walking. (present, progressive)*
 - o Tense
 - *I walked. I will walk. I had been walking. (past, future, past progressive)*
 - o Number and person
 - *I walk. They walk. (first person singular, third person plural)*
- **Problems**
- doubled consonants (sit, sitting)
- changing y to i (fly, flies)
- abbreviations (CAN0)
- naming word (Cook)
- wrong spelled word (Hare instead hair)
- unrecognized words:
 - o real words not in the dictionary
 - o names, etc that cannot be listed (names, account nos., library call nos)

iii. **Syntactic analysis system**

- Takes an i/p sentence and produces a representation of its grammatic structure.
- Grammar (mostly context free grammar) and parser are used.

iv. **Semantic Analysis System**

- Convert the syntactic representation into a meaning representation.
- It involves:
 - Word sense determination
 - Sentence level analysis

- Knowledge Representation
- v. **Pragmatic Analysis System**
 - Study how knowledge about the world and language conventions interacts with the literal meaning.
 - Depends upon context and facts about real world.
 - These aspects includes:
 - Pronouns and referring expressions.
 - Logical interfaces that can be drawn from meaning of a set of propositions.
 - Inference about what the producer of a sentence is trying to do.

Natural Language Analysis Techniques

- Two commonly used techniques used in natural language processing are
- i. **Keyword analysis or Pattern matching**
 - Accept the given sentence as input.
 - Segment the sentence.
 - Identify keywords in each segment.
 - If keyword is present
 - If only one keyword is present , give suitable reply as of keyword
 - If more the on keywords are present, prioritize them and give suitable reply as of keyword
 - If no keyword present in the segment , give a random reply.
- ii. **Syntactic driven parsing technique**
 - Words can fit together higher level units such as phrases, clauses and sentences.
 - Interpretations of larger groups of words are built up out of the interpretation of their syntactic constituent words or phrases.
 - Interpretation i/p done as a whole.
 - Obtained by application of grammar that determines what sentences are legal in the language that is being parsed.

NLP Problems

1. The same expression means different things in different context.
 - Where's the water? (Chemistry lab? Must be pure)
 - Where's the water? (Thirsty? Must be drinking water)
 - Where's the water? (Leaky roof? It can be dirty)
2. No natural language program can be complete because of new words, expression, and meaning can be generated quite freely.
 - I'll fax it to you
3. There are lots of ways to say the same thing.
 - Ram was born on October 11.
 - Ram's birthday is October 11.
4. Sentence and phrases might have hidden meanings
 - "Out of sight, out of mind"-> " invisible idiot"
 - "The spirit was willing but the flesh was weak" - > " the vodka was good, but the meat was bad"

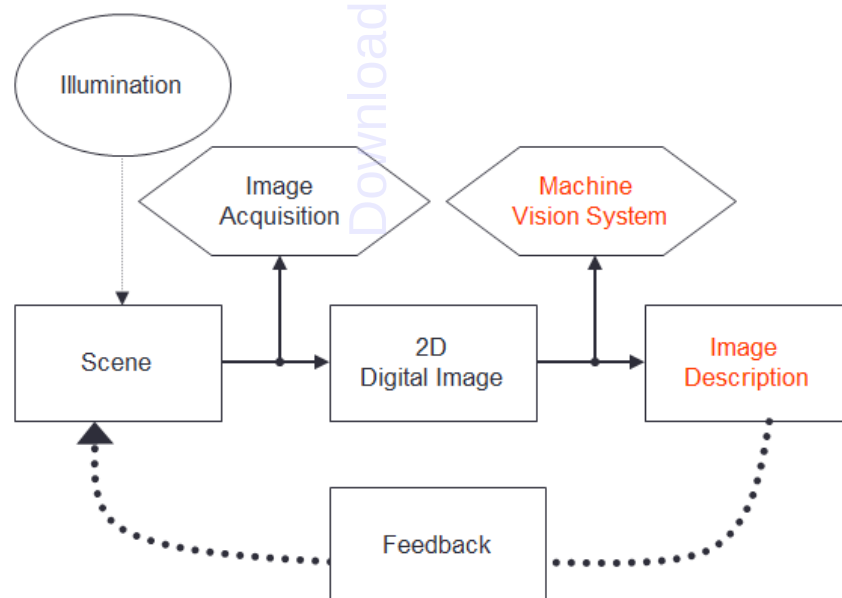
5. Problem due to syntax and semantics
6. Problem due to extensive use of pronouns. (semantic issue)
 - Eg. Ravi went to the supermarket. He found his favorite brand of coffee in rack. He paid for it and left.
 - It denotes??
7. Use of grammatically incorrect sentence
 - He rice eats. (syntax issue)
8. Use of conjunctions to avoid repetition of phrases cause problem in NLP
 - Eg. Ram and Hari went to restaurant. While Ram had a cup of coffee, Hari had tea.
 - Hari had a cup of tea.

Introduction to machine Vision

The goal of Machine Vision is to create a model of the real world from images

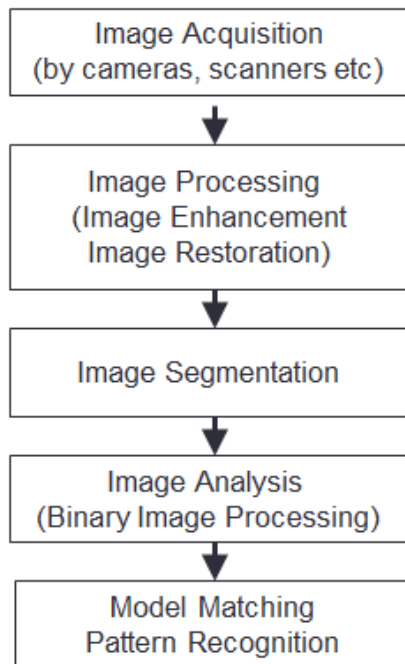
- A machine vision system recovers useful information about a scene from its two dimensional projections
- The world is three dimensional
- Two dimensional digitized image

Knowledge about the objects (regions) in a scene and projection geometry is required. The information which is recovered differs depending on the application such as Satellite, medical images etc.



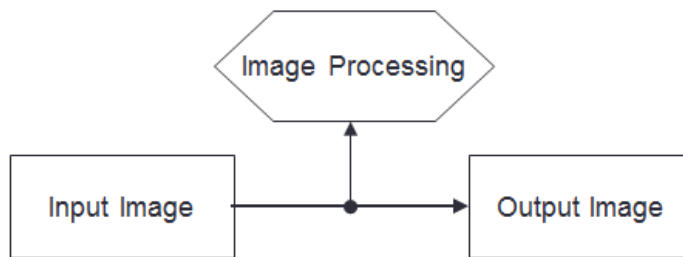
The goal of a machine vision system is to compute a meaningful description of the scene (e.g., object)

Machine Vision Stages



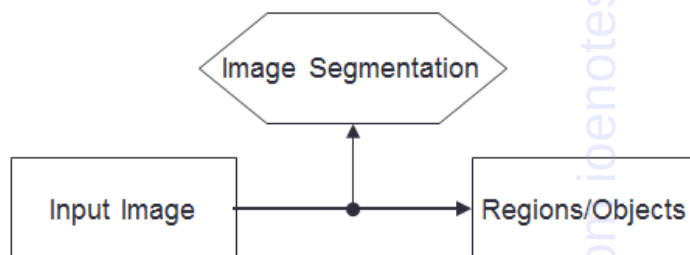
- Analog to digital conversion
- Remove noise/patterns, improve contrast
- Find regions (objects) in the image
- Take measurements of objects/relationships
- Match the above description with similar description of known objects (models)

Image Processing



Various procedures such as image enhancement (filtering, edge detection, surface detection, computation of depth) Image restoration (remove point/pattern degradation: there exist a mathematical expression of the type of degradation like e.g. Added multiplicative noise, sin/cos pattern degradation etc) takes place.

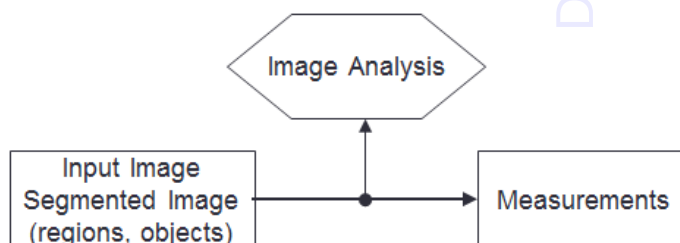
Image Segmentation



Classify pixels into groups (regions/objects of interest) sharing common characteristics.

- Intensity/Color, texture, motion etc.

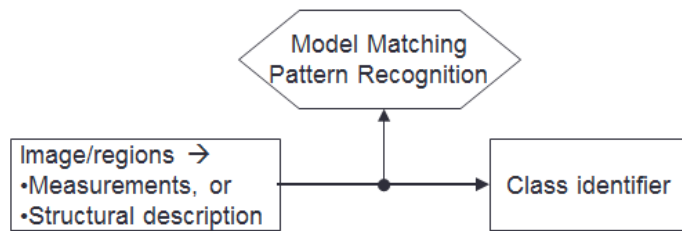
Image Analysis



Take useful measurements from pixels, regions, spatial relationships, motion etc.

- Grey scale / color intensity values;
- Size, distance;
- Velocity;

Pattern Recognition



Classify an image (region) into one of a number of known classes

- Statistical pattern recognition (the measurements form vectors which are classified into classes);
- Structural pattern recognition (decompose the image into primitive structures).

Digital Image Representation

Image: 2D array of gray level or color values

- Pixel: array element;
- Pixel value: arithmetic value of gray level or color intensity.

Gray level image: $f = f(x,y)$

- 3D image $f=f(x,y,z)$

Color image (multi-spectral)

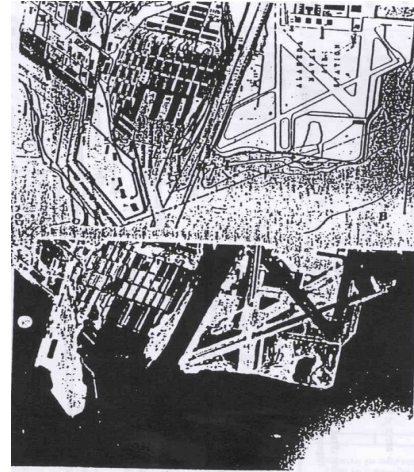
$$f = \{R_{\text{red}}(x,y), G_{\text{green}}(x,y), B_{\text{blue}}(x,y)\}$$

Machine Vision Applications

- Robotics
- Medicine
- Remote Sensing
- Meteorology
- Quality inspection

Remote Sensing

- Take images from high altitudes (from aircrafts, satellites).
- Find ships in the aerial image of the dock.
- Find if new ships have arrived.
- What kind of ships?



Medical Applications

- Assist a physician to reach a diagnosis.
- Construct 2D, 3D anatomy models of the human body.
- CG geometric models.
- Analyze the image to extract useful features.

